# File: 00-feature-catalogue.md

# Feature Catalogue - All Phases

## MediCore HIS - Master Feature Registry

> Single source of truth for every planned feature. Each feature is assigned a phase, RICE score, module, and implementation status.

---

## Phase Assignment Rules

- **P
billing

---

## Phase 1 - MVP Features (13 modules, ~25 features)

| ID | Feature | Module | RICE | Status |

|----|--

| F-017 | Public queue display (no auth) | Scheduling | 75 | Specified |

| F-01

---

## Phase 2 - Extended Platform (18 modules, ~80 features)

### Module Priority Order

1. Pa

16. Configurable Modules

17. B

| ID | Feature | Module | RICE | Complexity |

|----|--

| F-163 | In-session encounter documentation | Telemedicine | 80 | 4 |

| F-16

| F-280 | Embeddable booking widget (iframe/script) | Widget | 68 | 3 |

| F-28

---

## Phase 3 - Enterprise & Mobile (~35 features)

| ID | Feature | Module | RICE | Complexity |

|----|--

| F-403 | Leave management (request ? approve) | HR | 55 | 3 |

| F-40

---

## Summary

| Phase | Feature Count | New Modules | New DB Entities | New API Endpoints | New Screens |

|------

| Phase 2 | ~82 | 18 | ~25 | ~120 | ~55 |

| Pha

---

# File: 00-project-charter.md

# Phase 0 - Project Charter

### MediCore HIS · EPIOSO Studio · Oussama Zaki

---

## 1. Problem Statement

Outpatient clinics and mid-sized polyclinic networks - particularly in Morocco, MENA, and francophone Africa - are trapped between two bad options: overpriced, rigid enterprise EHRs (Athenahealth at 4-10% of collections, Tebra at $349-624/provider/month) and dangerously insecure CodeCanyon scripts ($39-199 one-time, built on aging PHP monoliths with zero compliance or modern AI). Neither option offers genuine multi-tenancy, event-driven workflow automation, or AI-native clinical intelligence at a price a solo practitioner or 5-doctor polyclinic can afford. The result: clinicians spend 2+ hours per 1 hour of patient care on manual documentation ("pajama time"), billing leakage reaches 5-15% of revenue, and cross-location operations run on disconnected spreadsheets.

---

## 2. Business Objective

Build and launch a multi-tenant SaaS Hospital Information System that combines a clinical operations core (appointments ? encounters ? prescriptions ? billing) with an AI copilot layer (text-to-SOAP expansion, differential suggestions, investigation recommendations) - priced aggressively enough to win solo practitioners and polyclinics away from legacy tools, while architected cleanly enough to scale toward hospitals and diagnostic centers in later phases.

### Measurable KPIs (Year 1 - Validation, Not Growth)

| KPI | Target | Timeframe |

|-----|-

| First real revenue | MRR ? $500/month | Month 12 |

| AI c

**Year 1 is validation, not scale.** No vanity metrics. If 3 clinics pay monthly and doctors actually use the AI copilot by Month 12, the product is validated.

---

## 3. Stakeholder Map

### Current State - Solo, Bootstrapped

| Name | Role | Responsibility | Status |

|------

### Path to Finding Help

- **C
know

---

## 4. RACI Matrix

| Decision Area | Responsible | Accountable | Consulted | Informed |

|------

| Pricing & business model | Oussama | Oussama | - | - |

**Escalation path for stuck problems**: AI pair programming ? ENSAM network ? Stack Overflow ? pivot to simpler solution.

---

## 5. Explicit Out-of-Scope (What We Are NOT Building in MVP)

### Hard No - Not in Phase 1

| Item | Reason |

|------

### Explicitly NOT Building Ever

| Item | Reason |

|------

---

## 6. Budget & Timeline

### Budget: $0 External - 100% Bootstrapped

| Cost Item | Solution | Monthly Cost |
|---|---|---|

**Funded by**: Freelance income from other EPIOSO projects.

**Rule**: No feature that costs >$20/month to run gets built before there's a paying customer.

### Timeline

| Milestone | Target | Duration |
|---|---|---|

**Developer capacity**: ~15-20 hours/week while studying at ENSAM. Build phases sized to 1-3 dev days each. One PR per day on build days.

---

## 7. Key Assumptions

| # | Assumption | Risk if Wrong |
|---|---|---|

| A2 | Doctors will adopt AI-generated SOAP notes with text input (not voice-only) | AI copilot becomes unused - monitor adoption rate closely |

| A3 |
than

---

## 8. Known Constraints

### Technical

- **S
Redis

### Legal & Compliance

- **No

### Organizational

- No

---

## 9. Open Questions Log

| # | Question | Status | Impact | Resolved |
|---|---|---|---|---|

---

## 10. Revision History

| Date | Change | Author |
|---|---|---|

---

**Founder Signature:** Oussama Zaki / EPIOSO Studio

**Dat

---

# File: 01-business-case.md

# Phase 1 - Business Case

## MediCore HIS - Investment, Revenue, and Risk Assessment

> Numbers sourced from: R1/Grok ($8K-25K estimates), R2/Gemini ($3K-3.5K bootstrapped MVP), R3/Perplexity ($5K-15K extended), his-predev project charter ($0 external, $120/mo burn rate).

---

# 1. Investment Required

### Scenario: Bootstrapped Solo Founder (Oussama's Reality)

MediCore is bootstrapped with $0 external funding. All costs are operational monthly expenses funded by freelance income from other EPIOSO projects.

#### Pre-Revenue Costs (Months 1-8)

| Category | Cost | Notes |

|------

This is **not** the $15K-25K that research sources cite for a typical health-tech SaaS launch - because Oussama is the entire team. There are no salaries, no contractors, no designers, no compliance consultants in Phase 1.

#### Cost of First Customer Live (Month 8 onwards)

| Category | Monthly Cost |

|------

**Break-even**: 3 Solo customers ($39/mo × 3 = $117/mo) or 1 Clinic customer ($149-199/mo). Achievable by Month 9-10.

---

# 2. Revenue Model

### Pricing Tiers (Locked - Aggressive Undercut Strategy)

| Tier | Price | Target | Includes |
|---|---|---|---|

### Why This Pricing Works

1. **Undercuts every mid-market competitor by 85-92%**: A 5-doctor polyclinic pays $199/mo with MediCore vs $1,500-2,500/mo with Tebra or DrChrono

2. **P
penal

### Revenue Projections

| Time | Customers | Mix | MRR | ARR |
|---|---|---|---|---|

**Conservative**: These projections assume zero Enterprise customers and no investor-funded growth.

---

## 3. ROI & Payback Analysis

### For Oussama (the builder)

| Metric | Value |
|---|---|

|--------|-------|

| **T
$14,4

### For the Customer (Dr. Youssef - Solo)

| Metric | Value |

|------

### For the Customer (Dr. Amina - Clinic)

| Metric | Value |

|------

---

## 4. Risk / Reward Summary

### Significant Risks

| # | Risk | Probability | Impact | Mitigation |

|---|---

| R4 | **Solo founder burnout** - 15-20 hr/week alongside ENSAM studies | High | High | Sustainable pace rules; 1 PR/day; hard stop on 1-week debugging sessions. Seek co-builder early |

| R5 |
CMI (

### Rewards

| Reward | Mechanism | Timeline |

|------

---

## 5. Go / No-Go Recommendation

### ? GO - Conditional on Validation

**The case for building is strong:**

- The

**Conditions for continued investment:**

1. **M

4. **Month 12**: MRR ? $500

If conditions 1-2 aren't met by Month 8, pause and reassess whether the product concept or execution is the problem. If condition 4 isn't met by Month 14, consider pivoting to a different market segment (e.g., diagnostic labs instead of clinics) or exploring a B2B2B model (selling through clinic management consultancies).

---

## 6. Financial Model Summary

```
```

Year

Year 2 Scenario (Growing)

????

Year 3 Scenario (Scaled)

????

---

# File: 01-competitive-analysis.md

# Phase 1 - Competitive Analysis

## MediCore HIS - Competitive Landscape

> Data sourced from: R1/Grok (pricing ranges), R2/Gemini (detailed feature comparison), R3/Perplexity (54 citations with per-vendor analysis), G2/Capterra reviews.

---

# 1. Competitive Matrix - Direct Competitors

## Tier 1: Enterprise Incumbents (Overpriced for Our Market)

#### Athenahealth (athenaOne)

| Dim

#### eClinicalWorks (eCW)

| Dim

---

## Tier 2: Mid-Market (Our Direct Competition)

#### Tebra (formerly Kareo + PatientPop)

| Dim

| **AI Story** | Near-zero. No native AI clinical features at time of analysis |

| **M
is not

#### DrChrono (now EHR by DrChrono)

| Dim

---

### Tier 3: Budget Market (What We Must Destroy)

#### CodeCanyon Scripts (Smart Hospital, Clinic365, HospitalRun)

| Dim

---

## 2. Pricing Comparison Table

| Vendor | Solo / 1 Provider | Clinic (3-5 Providers) | Enterprise |
|---|---|---|---|

**MediCore's pricing advantage**: A 5-doctor polyclinic pays $199/mo total with MediCore vs $1,500-2,500/mo with mid-market competitors. That's **87-92% cheaper** with a stronger AI story.

---

## 3. Indirect Competitors & Adjacent Threats

| Type | Examples | Threat Level | Why Not a Killer |
|---|---|---|---|

---

## 4. Positioning Map

```
```

dangerous) ? ? eClinicalWorks

**The white space**: MediCore occupies the only quadrant with BOTH aggressive pricing AND native AI - a position no current product serves.

---

## 5. Competitive Moats (What Becomes Harder to Replicate Over Time)

| Moat | Mechanism | Timeline |
|------

---

## 6. Key Takeaway

The competitive landscape is polarized: expensive enterprise tools with bolted-on AI, or cheap scripts with zero intelligence and catastrophic security. MediCore slots into the massive gap between them - offering enterprise-grade multi-tenancy and native AI at CodeCanyon-adjacent pricing. The positioning is defensible because incumbents are structurally incapable of matching our price point, and CodeCanyon scripts are structurally incapable of matching our architecture.

---

# File: 01-market-opportunity.md

# Phase 1 - Market Opportunity

### MediCore HIS - Market Sizing & Timing Analysis

> Data triangulated from: Grok (R1), Gemini (R2), Perplexity (R3 - 54 sourced citations), Grand View Research, Precedence Research, Verified Market Research, Fortune Business Insights.

---

## 1. TAM - Total Addressable Market

### Global EHR / HIS Market

| Source | Estimate (2025) | Projection | CAGR |
|---|---|---|---|

**Working TAM**: **~$30-36B** (global cloud-based EHR/HIS, 2025).

Cloud dominates: **81.1% share** of EHR deployments are now cloud-hosted (Source: R2/Gemini citing Grand View, Precedence). The on-premise segment is actively shrinking.

### AI in Healthcare - Mega-Trend

The broader AI in healthcare market is projected at **$45.2B (2023) ? $148.4B (2029)** at 20.5% CAGR. The AI-powered clinical documentation sub-segment alone crossed **$3.2B** in 2024 (Source: R3/Perplexity citing Markets and Markets, Polaris). This creates a direct tailwind for MediCore's AI copilot positioning.

---

## 2. SAM - Serviceable Addressable Market

### Ambulatory EHR Market (Our Segment)

MediCore targets the ambulatory (outpatient) market: solo practitioners, small-to-medium clinics, and polyclinic networks with 1-15 providers.

| Source | Ambulatory EHR Estimate | Notes |
|---|---|---|

**Working SAM**: **$5.7-6.8B** (global ambulatory cloud EHR, 2025).

### Why Ambulatory, Not Hospital First

- **Lower sales friction**: Solo and small-group practices can adopt SaaS without board approval or committee review

- **Fa

---

## 3. SOM - Serviceable Obtainable Market

### Realistic Year 1-3 Targets for a Solo Bootstrapped Founder

| Scenario | Clinics | Blended ARPU | ARR |

|------

This represents **<0.01% of SAM** - an achievable target that requires no market-moving events.

**First geography**: Morocco (Casablanca, then Rabat, Marrakech). Expand to: Tunisia, Côte d'Ivoire, Senegal, then Gulf states (UAE, Saudi).

**Revenue model**: Monthly SaaS subscription per organization (not per-provider), making multi-doctor clinics more attractive than seat-based competitors.

---

## 4. Market Growth Drivers & Timing

### Why Now - Structural Shifts Enabling MediCore

#### 1. The "Pajama Time" Crisis (Clinical Burnout)

- Doc
Medi

- **MediCore implication**: AI copilot is not a nice-to-have - it's the primary acquisition hook

#### 2. AI Commoditization (Cost Collapse)

- GPT

#### 3. Cloud Security vs. Ransomware Targeting

- 56%

#### 4. Value-Based Care & Documentation Pressure

- Sma

#### 5. MENA & Africa Digital Health Investment

- Mor

---

## 5. Timing Risk Assessment

| Factor | Risk Level | Mitigation |

|------

---

## 6. Key Takeaway

The market timing is ideal: AI costs have collapsed, clinical burnout is at crisis levels, and the MENA/Africa ambulatory segment is underserved by modern SaaS. MediCore doesn't need to create demand - it needs to capture existing, frustrated demand with a product that is **cheaper than Tebra, smarter than Kareo, and safer than CodeCanyon**.

---

# File: 01-user-personas.md

# Phase 1 - User Personas

## MediCore HIS - Target User Profiles

> Personas synthesized from: R1/Grok (behavioral profiles), R2/Gemini (psychographic analysis, tech comfort mapping), R3/Perplexity (MENA-specific names, regional context), his-predev (Moroccan clinic realities).

---

## Persona 1: Dr. Youssef El Amrani - The Solo Practitioner

### Profile

| Field | Detail |

|------

### Goals

1. **L

### Frustrations

- Spe
("tem

- Tried a CodeCanyon script once - it crashed, data was lost, no one answered support emails

- Athe

### Tech Comfort

**Low
frustr

### How He Discovers MediCore

- Wor

### What Makes Him Stay

- AI c

### Implications for MediCore Design

- **Or

---

## Persona 2: Dr. Amina Benabdallah - The Polyclinic Director

### Profile

| Field | Detail |

|------

| **Monthly tech budget** | $200-500 |

| **La

## Goals

1. **S
billing

## Frustrations

- Can
no his

## Tech Comfort

**Mid
analy
decis

## How She Discovers MediCore

- Sea

## What Makes Her Stay

- One

## Implications for MediCore Design

- **Mu

- **Pricing at $149-199/mo flat** (not per-provider) - she'll compare against per-seat alternatives

- **O
sessi

---

## Persona 3: Fatima Zahra Idrissi - The Receptionist

### Profile

| Field | Detail |

|------

### Goals

1. **F

### Frustrations

- The

### Tech Comfort

**Low
Need

### How She Uses MediCore

- She

- If she can't figure it out in that session, she'll quietly resist using it

### What Makes Her Stay

- Pati

### Implications for MediCore Design

- **Re
rest c

---

## Persona 4: Karim Berrada - The IT-Minded Office Manager

### Profile

| Field | Detail |

|------

### Goals

1. **S
calls

5. **Predictable costs** - no usage-based billing surprises

### Frustrations

- The

### How He Discovers MediCore

- Foru

### What Makes Him Stay

- **Sa

### Implications for MediCore Design

- **AP

---

## Persona Priority for MVP

| Persona | Priority | Reason |

|------

---

# File: 02-PRD.md

# Phase 2 - Product Requirements Document (PRD)

## MediCore HIS - MVP Feature Specification

---

## 1. Product Vision

MediCore is a multi-tenant SaaS Hospital Information System that enables solo practitioners and polyclinic networks to manage their entire clinical workflow - from patient registration through encounter documentation to billing - with an AI copilot that transforms shorthand notes into structured SOAP documentation, reducing physician documentation time by 30-45 minutes per day.

### Target Users (MVP)

- **Do

### Success = A Doctor Finishes a Full Day's Documentation Before Leaving the Clinic

---

## 2. Feature List with RICE Scores

> RICE = (Reach × Impact × Confidence) ÷ Effort

> Rea

### Core Features (MVP - Must Ship)

| # | Feature | Reach | Impact | Confidence | Effort (wks) | RICE | Priority |

|---|---

| F05 | **Doctor schedule management** | 8 | 2 | 100% | 1.5 | 10.7 | P0 |

| F06

**Not in MVP (Phase 2+)**

| Feature | Phase | Reason for Deferral |

|------

---

## 3. User Flows (Narrative)

### Flow 1: New Clinic Onboarding

1. Fo

### Flow 2: Patient Visit (End-to-End)

1. **F

### Flow 3: Multi-Facility Admin Day

1. Ad

---

## 4. Non-Goals (Explicit)

- MediCore is NOT a medical device - it does not make clinical decisions

- The

---

## 5. Success Metrics per Feature

| Feature | Success Metric | Target |

|------

---

## 6. Open Questions

| # | Question | Impact | Status |

|---|---

---

# File: 02-PRD-phase2-3.md

# Phase 2 & 3 - Product Requirements Document (Expansion)

# MediCore HIS - Full Product Roadmap PRD

> This document extends `02-PRD.md` (MVP features). All Phase 1 features remain unchanged.

---

## Phase 2 Features - RICE Scored & Ordered

### P2.1 Patient Portal

**Goal**: Self-service patient access to medical records, appointments, invoices, and consent management. Phone-based OTP authentication (no passwords).

| Feature | Reach | Impact | Confidence | Effort | RICE |
|---|---|---|---|---|---|

**User Flow - Patient Self-Service Booking**:

```

**User Flow - Patient Invoice Payment**:

```

---

### P2.2 Lab Module

**Goal**: Full lab workflow from order placement (by doctors during encounters) through specimen collection, result entry, auto-flagging of abnormal values, result authorization, and billing integration.

| Feature | Reach | Impact | Confidence | Effort | RICE |
|---

**User Flow - Lab Order to Result**:

```

Lab tech sees order in /lab/queue ? Marks "Specimen Collected"

? Ent

Supervising doctor reviews ? Approves ? Results appear in encounter timeline

? Lab

---

### P2.3 Pharmacy Module

**Goal**: Drug catalog, pharmacy inventory with batch/expiry tracking, prescription-to-dispensing workflow, pharmacist verification, stock alerts, and billing integration.

| Feature | Reach | Impact | Confidence | Effort | RICE |
|---
```

| Purchase orders (restock requests) | 2 | 3 | 4 | 3 | 8 |

| Sup

**User Flow - Prescription to Dispensing**:

```

Pharmacist sees Rx in /pharmacy/queue ? Reviews drug + dosage

? Che

If insufficient stock ? Alert shown ? Pharmacist flags as "Pending Restock"

? Pur

---

### P2.4 Telemedicine

**Goal**: Video consultations linked to appointments using Daily.co or LiveKit. Includes waiting room, in-session encounter documentation, and telemedicine-specific billing codes.

| Feature | Reach | Impact | Confidence | Effort | RICE |

|------

---

### P2.5 IPD Module (Inpatient)

**Goal**: Full inpatient workflow - ward/bed management, patient admission (from OPD encounter or direct), bed allocation with conflict prevention, nurse station dashboard, daily round notes, discharge summary, and bed charges billed per day.

| Feature | Reach | Impact | Confidence | Effort | RICE |

|------
```

| IPD round notes (daily progress) | 3 | 4 | 4 | 3 | 16 |

| Disc

---

### P2.6 Inventory Module

**Goal**: Track consumables and equipment across facilities. Stock management with receive/issue/adjust, low-stock alerts, purchase orders with approval workflow.

| Feature | Reach | Impact | Confidence | Effort | RICE |

|------

---

### P2.7 Radiology Module

**Goal**: Radiology orders from encounters, template-based report entry, authorization workflow, optional DICOM/Orthanc viewer integration, billing integration.

| Feature | Reach | Impact | Confidence | Effort | RICE |

|------

---

### P2.8 Internal Messaging

**Goal**: Staff-to-staff messaging within the org. Direct messages, group conversations (per department/facility), message notifications, and ability to link conversations to patient context.

| Feature | Reach | Impact | Confidence | Effort | RICE |
|---|---|---|---|---|---|

---

### P2.9-P2.18 Remaining Modules (Summary)

| Module | Key Features | Total Endpoints | Screens |
|---|---|---|---|

---

## Phase 3 Features - RICE Scored

### P3.1 HR Module

| Feature | RICE | Summary |
|---|---|---|

### P3.2 Workflow Automation Engine

| Feature | RICE | Summary |
|---|---|---|

|---------|------|---------|

| Sim

### P3.3 React Native Mobile Apps

| App | Key Screens | Reuses API |

|-----|-

### P3.4 HL7 FHIR API

| Resource | Maps To | Notes |

|------

### P3.5 AI Advanced

| Feature | RICE | Summary |

|------

### P3.6 Ambulance Management

| Feature | RICE | Summary |

|------

| Dispatch | 6 | Link ambulance to patient, track status |

| Billi

---

## Non-Goals (Not Planned for Any Phase)

| Feature | Reason |

|------

---

# File: 02-SRS.md

# Phase 2 - Software Requirements Specification (SRS)

## MediCore HIS - Functional & Non-Functional Requirements

---

## 1. Functional Requirements

### 1.1 Authentication & Authorization

| ID | Requirement | Testable Criterion |

|----|--

| FR-005 | The system shall enforce role-based permissions using `@Permissions()` decorators on protected endpoints | A RECEPTIONIST calling a DOCTOR-only endpoint receives 403 |

| FR-
A use

### 1.2 Tenancy

| ID | Requirement | Testable Criterion |

|----|--

### 1.3 Patient Management

| ID | Requirement | Testable Criterion |

|----|--

### 1.4 Scheduling

| ID | Requirement | Testable Criterion |
|---|---|---|

## 1.5 Encounters (EMR Core)

| ID | Requirement | Testable Criterion |
|---|---|---|

| FR-049 | The system shall support paginated encounter history per patient (summary view) | `GET /patients/:id/encounters?page=1&limit=10` returns encounters with visit_date, status, doctor |

### 1.6 Billing

| ID | Requirement | Testable Criterion |
|---|---|---|

### 1.7 AI Copilot

| ID | Requirement | Testable Criterion |
|---|---|---|

### 1.8 Communication

| ID | Requirement | Testable Criterion |
|---|---|---|

---

## 2. Non-Functional Requirements

### 2.1 Performance

| ID | Requirement | Target |
|---|---|---|

### 2.2 Availability & Reliability

| ID | Requirement | Target |
|---|---|---|

### 2.3 Scalability

| ID | Requirement | Target |
|---|---|---|

| NFR-021 | Concurrent encounters per facility | Up to 20 simultaneous |

| NFR

### 2.4 Security

| ID | Requirement | Target |

|----|--

### 2.5 Usability

| ID | Requirement | Target |

|----|--

---

## 3. External Interface Requirements

| Interface | Protocol | Direction | Purpose |

|------

---

## 4. System Constraints

| Constraint | Description |

|------

---

# File: 02-SRS-phase2-3.md

# Phase 2 & 3 - Software Requirements Specification (Expansion)

## MediCore HIS - Functional Requirements FR-100 to FR-350

> Extends `02-SRS.md` (FR-001 to FR-083, NFR-001 to NFR-022). Phase 1 requirements unchanged.

---

## Patient Portal - FR-100 Series

| ID | Requirement | Priority | Testable Criterion |

|----|--

| FR-109 | Portal SHALL process Stripe payments and update invoice status | Must | Payment ? invoice PAID within 10s |

| FR-
frequ

---

## Lab Module - FR-120 Series

| ID | Requirement | Priority | Testable Criterion |

|----|--

---

## Pharmacy Module - FR-140 Series

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

---

## Telemedicine - FR-160 Series

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

---

## IPD Module - FR-180 Series

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

---

## Radiology - FR-220 Series

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

---

## Messaging - FR-240 Series

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

| FR-244 | Unread message count SHALL update in real-time | Must | Badge count via WebSocket |

---

## Triage - FR-260 Series

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

---

## Ambient Audio AI - FR-270 Series

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

---

## Platform Features - FR-280 to FR-350

| ID | Requirement | Priority | Testable Criterion |
|---|---|---|---|

| FR-292 | Campaign analytics SHALL show delivered/failed/opt-out counts | Should | Counts accurate |

| FR-
comp

---

## Phase 3 - FR-400 to FR-460 (Summary)

| ID | Requirement | Module |

|----|--

---

## Updated NFRs for Phase 2/3

| ID | Requirement | Target |

|----|--

| NFR-023 | WebSocket messaging latency SHALL be <1 second P95 | Messaging and queue |

---

# File: 02-user-stories.md

# Phase 2 - User Stories

## MediCore HIS - Acceptance-Criteria-Driven User Stories

> Format: **As a** [persona], **I want to** [action] **so that** [benefit].

---

## Authentication & Onboarding

### US-001: Clinic Onboarding

**Acceptance criteria:**

### US-002: Login & Session

**Acceptance criteria:**

- Given I enter wrong password, when I click login, then I see "Invalid email or password" - not "User not found" (no email enumeration)

- Giv
re-log

### US-003: Multi-Facility Access

**As
see o

**Acceptance criteria:**

- Giv
listed

---

## Patient Management

### US-010: Patient Registration

**As
the lo

**Acceptance criteria:**

- Giv
creat

### US-011: Patient Search

**As
that**

**Acceptance criteria:**

- Give
last n

- Given a patient belongs to a different organization, when I search, then they never appear in my results

### US-012: Patient Allergy Management

**As
drug.

**Acceptance criteria:**

- Giv
(drug

---

## Scheduling

### US-020: Schedule Management

**As
can c

**Acceptance criteria:**

- Give
my av

### US-021: Appointment Booking

**As
patie

**Acceptance criteria:**

- Give

### US-022: Check-In & Queue

**As
waitin

**Acceptance criteria:**

- Giv
T-042

### US-023: No-Show Detection

**As
recep

**Acceptance criteria:**

- Give
the a

---

## Encounters (EMR Core)

### US-030: Open Encounter

**As
the vi

**Acceptance criteria:**

- Giv
creat

### US-031: Vitals Recording

**As
part o

**Acceptance criteria:**

- Give
need

### US-032: SOAP Note with AI Copilot

**As
**so t

**Acceptance criteria:**

- Give
the co

### US-033: Diagnosis Entry

**As
recor

**Acceptance criteria:**

- Give
with c

### US-034: Prescription Writing

**As
comp

**Acceptance criteria:**

- Give

### US-035: Encounter Sign-Off

**As
clinic

**Acceptance criteria:**

- Give
appe

---

## Billing

### US-040: Auto-Invoice on Sign-Off

**As
that**

**Acceptance criteria:**

- Give
draft

### US-041: Invoice Management

**As
accur

**Acceptance criteria:**

- Give
auton

### US-042: Online Payment

**As
carry

**Acceptance criteria:**

- Given the receptionist clicks "Pay Online", when the Stripe payment form loads, then the patient can enter card details

- Give
times

---

## AI Copilot

### US-050: Patient Summary

**As
that**

**Acceptance criteria:**

- Give
summ

### US-051: Differential Suggestions

**As
text *

**Acceptance criteria:**

- Giv
condi

---

## Communication

### US-060: Appointment SMS

**As
befor

**Acceptance criteria:**

- Giv
receiv

- Given my appointment is tomorrow at 14:00, when the scheduled job runs today at 14:00, then I receive a reminder SMS

- Give
failure

---

## Admin & Analytics

### US-070: Revenue Dashboard

**As
which

**Acceptance criteria:**

- Give
by fa

### US-071: Audit Trail

**As
handl

**Acceptance criteria:**

- Giv
("pati

---

# File: 02-user-stories-phase2-3.md

# Phase 2 & 3 - User Stories

## MediCore HIS - Extended User Stories with Acceptance Criteria

> Extends `02-user-stories.md` (US-001 to US-025). User stories numbered US-100+ for Phase 2, US-200+ for Phase 3.

---

## Patient Portal (US-100 to US-111)

### US-100: Patient OTP Login

**As
my re

**Acceptance Criteria:**

- **Gi

### US-101: View My Medical Records

**As a

**Acceptance Criteria:**

- **Gi

### US-102: Book Appointment from Portal

**As a

**Acceptance Criteria:**

- **Gi

### US-103: Pay Invoice from Portal

**As
the cl

**Acceptance Criteria:**

- **Gi

- **And** on successful payment, the invoice status changes to PAID

- **Ar

### US-104: Cancel Appointment

**As
other

**Acceptance Criteria:**

- **Gi

### US-105: Manage Privacy Consent

**As
data.

**Acceptance Criteria:**

- **Gi

---

## Lab Module (US-120 to US-128)

### US-120: Order Lab Test from Encounter

**As
notifie

**Acceptance Criteria:**

- **Gi

### US-121: Enter Lab Results

**As a** lab technician, **I want to** enter test results and have abnormal values flagged automatically, **so that** doctors are alerted immediately.

**Acceptance Criteria:**

- **Gi

### US-122: Authorize Lab Results

**As
recor

**Acceptance Criteria:**

- **Gi

### US-123: View Lab Results in Encounter

**As
one p

**Acceptance Criteria:**

- **Gi

---

## Pharmacy Module (US-140 to US-147)

### US-140: Dispense Medication

**As
decre

**Acceptance Criteria:**

- **Gi

- **Then** stock decrements by 30 tablets for the selected batch

- **Ar

### US-141: Low Stock Alert

**As*
resto

**Acceptance Criteria:**

- **Gi

### US-142: Purchase Order Workflow

**As
appro

**Acceptance Criteria:**

- **Gi

---

## Telemedicine (US-160 to US-163)

### US-160: Video Consultation

**As

**Acceptance Criteria:**

- **Gi

---

## IPD Module (US-180 to US-189)

### US-180: Admit Patient

**As
inpati

**Acceptance Criteria:**

- **Gi

### US-181: Nursing Station Dashboard

**As
any c

**Acceptance Criteria:**

- **Gi

### US-182: Discharge Patient

**As
follow

**Acceptance Criteria:**

- **Gi

---

## Radiology (US-220 to US-222)

### US-220: Order Radiology Study

**As a

**Acceptance Criteria:**

- **Gi

### US-221: Enter Radiology Report

**As a

**Acceptance Criteria:**

- **Gi

---

## Messaging (US-240 to US-242)

### US-240: Send Direct Message

**As
patien

**Acceptance Criteria:**

- **Gi

---

## Triage (US-260)

### US-260: Symptom-Based Triage

**As
wheth

**Acceptance Criteria:**

- **Gi

- **When** they enter symptoms ("chest pain, shortness of breath")

- **Th

---

## CMS (US-320)

### US-320: Build Clinic Marketing Page

**As
book

**Acceptance Criteria:**

- **Gi

---

## Phase 3 User Stories (Summary)

### US-200: Workflow Rules

**As
(Diab

### US-210: HR Attendance

**As*
that**

### US-220: Payroll

**As*
salary

### US-230: Patient Mobile App

**As
conve

**US-240: Doctor Mobile App**

**As
prepa

**US-250: FHIR Interoperability**

**As*
integ

---

# File: 03-architecture-overview.md

# Phase 3 - Architecture Overview

## MediCore HIS - System Architecture

---

## 1. System Context

MediCore HIS is a multi-service SaaS platform with three runtime components, three data stores, and five external integrations:

```mermaid
C4Co

    Person(doctor, "Doctor", "Records encounters, prescriptions, uses AI copilot")
Pe

    System(medicore, "MediCore HIS", "Multi-tenant SaaS HIS with AI copilot")

    System_Ext(openai, "OpenAI API", "GPT-4o / GPT-4o-mini for clinical AI")
Sy

    Rel(doctor, medicore, "Uses", "HTTPS")
Re
```

Rel(medicore, sendgrid, "Sends email", "HTTPS")

Re

---

## 2. Major Components and Responsibilities

| Component | Technology | Responsibility | Port |

|------

---

## 3. Data Flow Summary

### Write Path: Doctor Signs Off an Encounter

```mermaid

seque

D->>FE: Click "Sign Off"

FE
```

API->>Redis: Publish clinics.encounter.signed_off

Re

### Read Path: AI Copilot Generates Patient Summary

```mermaid

seque

D->>FE: Open encounter ? sidebar loads

FE

---

## 4. Third-Party Integration Map
```

| Service | Used By | Purpose | Auth Method | Cost Model |
|---|---|---|---|---|

---

## 5. Communication Patterns

### Module-to-Module Communication

Modules inside NestJS communicate **exclusively via Redis events**. No module imports another module's service directly.

| Event | Publisher | Subscriber(s) | Action |
|---|---|---|---|

### Event Envelope Format

```json
{
```

"encounter_id": "6612abc...",

"pa

---

## 6. Deployment Overview

```mermaid

graph

subgraph "Railway ($15/mo total)"

A

subgraph "MongoDB Atlas"

subgraph "External Services"

FE -->|REST + SSE| API

FE
```

API --> ST

AP

**Total cost**: ~$20/mo (dev) ? ~$80-120/mo (first clinic live)

---

# File: 03-architecture-phase2-3.md

# Phase 2 & 3 - Architecture Expansion

## MediCore HIS - Extended System Architecture (All Phases)

> Extends `03-architecture-overview.md`. MVP architecture remains unchanged. This document adds Phase 2/3 modules, data stores, events, and service topology.

---

## 1. Full System Container Diagram (All Phases)

```mermaid

graph

subgraph "Frontend - Vercel"

subgraph "Mobile - App Stores (P3)"
```

```
end

subgraph "Backend - Railway"

subgraph "Data Stores"

subgraph "External Services"

DOC & REC & ADM & NUR & LAB & PHR & RAD -->|HTTPS| WEB

WEB & PORTAL & CMS & PATAPP & DOCAPP -->|REST + SSE| API

API --> PG & MG & RD

API --> TW & SG & ST & R2 & DAILY
```

---

## 2. NestJS Module Map (All Phases)

```mermaid
graph

    subgraph "Phase 2 Modules"

    subgraph "Phase 3 Modules"

    subgraph "Events (Redis Pub/Sub)"

    ENC -->|encounter.signed_off| EVT
```

EVT -->|subscribe| BILL

EV

---

## 3. Event Bus Contracts (Phase 2/3 Additions)

### New Event Channels

| Channel | Event | Payload | Published By | Consumed By |

|------

| `portal.*` | `portal.payment.completed` | `{ invoice_id, payment_id, method: "stripe", amount }` | Portal | Billing (update status) |

| `tele

- |

---

## 4. Data Store Ownership (Phase 2/3 Additions)

### New PostgreSQL Tables (Phase 2)

| Table | Module | Purpose |

|------

### New PostgreSQL Tables (Phase 3)

| Table | Module | Purpose |

|------

| `leave_requests` | HR | Leave applications with approval status |

| `pay

**New MongoDB Collections (Phase 2)**

| Collection | Module | Purpose |

|------

**New MongoDB Collections (Phase 3)**

| Collection | Module | Purpose |

|------

---

## 5. Service Communication Matrix

```mermaid

graph
```

```
    API -->|HTTP| TWILIO["Twilio API"]

    subgraph "Asynchronous (Redis Pub/Sub)"

    subgraph "Real-time (SSE/WebSocket)"

    subgraph "Background (Cron/Scheduled)"
```

---

## 6. New RBAC Roles (Phase 2/3)

| Role | Phase | Primary Permissions |
|---|---|---|

---

## 7. Deployment Topology (Phase 2)

```mermaid
graph

    subgraph "Railway"

    subgraph "MongoDB Atlas"

    subgraph "Cloudflare"

    subgraph "External SaaS"

    WEB_PROD & PORTAL_PROD & CMS_PROD --> API_PROD
```

**Phase 2 cost estimate**: ~$150-200/month (up from $102-117/month MVP).

---

# File: 03-C4-diagrams.md

# Phase 3 - C4 Diagrams

## 1. C4 Context Diagram (Level 1)

Shows MediCore as a single system interacting with users and external services.

```mermaid
graph

    subgraph "MediCore HIS"

    subgraph "External Services"

    DOC -->|HTTPS| MC
RE

    MC -->|HTTPS| OAI
MC
```

## 2. C4 Container Diagram (Level 2)

Shows the internal runtime containers and data stores.

```mermaid
graph

  subgraph "MediCore HIS Platform"

    subgraph "Backend - Railway"

3001"

    subgraph "Data Stores - Railway + Atlas"

  subgraph "External"

  U -->|HTTPS| WEB

WE

  API --> PG

AP
```

---

## 3. C4 Component Diagram (Level 3) - NestJS Core API

Shows the internal modules of the main backend service.

```mermaid
graph

    subgraph "Domain Modules"

    subgraph "Infrastructure"

  JWT --> AUTH
PE

  AUTH --> PRISMA
TN

  PAT --> MONGOOSE
SC

  SCHED --> REDIS
EN
```

```

---

## 4. C4 Component Diagram (Level 3) - FastAPI AI Sidecar

```mermaid
graph

    subgraph "Routers"
Route

    subgraph "Services"

    subgraph "Prompts"

JWTV --> COP
RA

CTX -->|GET /internal/*| NESTJS["NestJS API"]
LL

---

## 5. Data Ownership Map

Each NestJS module owns specific data stores. No module crosses data boundaries.

```mermaid
graph

    subgraph "MongoDB (Mongoose)"

    subgraph "Redis"

    AUTH_MOD["Auth Module"] --> PG_AUTH
TN
```

> **Exception**: The Scheduling module touches both PostgreSQL (doctor_schedules) and MongoDB (appointments). This is intentional - schedules are structured/relational (JOINed with Users), while appointments are document-like (embedded status_history). The module internally uses two repository

classes: `sched-pg.repository.ts` and `sched-mongo.repository.ts`.

---

# File: 03-tech-stack-decisions.md

# Phase 3 - Tech Stack Decisions

## MediCore HIS - Technology Choices and Rationale

---

### Final Stack Table

| Layer | Technology | Version | Why This | Why Not the Alternative |
|---|---|---|---|---|

| **AI Model** | OpenAI GPT-4o / GPT-4o-mini | Latest | Best clinical text accuracy among available LLMs. GPT-4o-mini for cost-efficient daily use, GPT-4o for complex differentials | Claude (Anthropic - excellent but API availability varies by region), Gemini (Google - strong but less clinical validation), open-source (Llama - requires GPU hosting, $100+/mo) |

| **Fi
docu
Supa

---

## Monorepo Structure

```

medi
```

??? .github/

??

**Why pnpm monorepo?**

- Sha

---

### Key Design Decisions Summary

| Decision | Rationale | ADR |

|------

---

## File: 04-information-architecture.md

## Phase 4 - Information Architecture

### MediCore HIS - Site Map, URL Structure & Navigation

---

### 1. Application Shell Architecture

MediCore uses **4 shell layouts**, each with its own sidebar navigation:

```mermaid
```

```
graph TB
    RO
    AUTH --> LOGIN["/login"]
    AU
    ADMIN --> DASH["/admin/dashboard"]
    AD
    CLINICAL --> PAT_LIST["/clinical/patients"]
    CL
    OPS --> OPS_PAT["/ops/patients"]
    OP
    PUBLIC --> QUEUE_TV["/queue-display/[facilityId]"]
```

---

## 2. URL Structure Table

| Route | Shell | Role(s) | Description |
|------

| `/admin/facilities` | Admin | ORG_ADMIN | Facility CRUD, settings per facility |

| `/ad
etc.) v

---

## 3. Navigation Hierarchy

### Admin Sidebar

```

### Clinical Sidebar
```

```

? Pat

**Operations Sidebar**

```

**Facility Switcher (always visible in top nav)**

```

---

## 4. Content Groupings

| Group | Pattern | Rationale |

|------

---

# File: 04-pages-bible-part1.md

# Pages Bible - Part 1: Auth, Onboarding & Global Layout

> Definitive page-by-page specification for MediCore HIS.

> C
`DES
```

> Each page lists: route, role access, API calls, layout, components, states, UX rules, and relations to other pages.

---

## 1. Authentication & Session Pages

### PG-001 · Login

| Property | Value |
|---|---|

**Layout:**

```
```

**States:**

| Stat

**UX Rules:**

- ? Ne

**Audit gap found:** Our current `POST /auth/login` returns user but doesn't track device info. **Fix:** Add `user_agent` and `ip` to the login audit log (server-side, no API change needed).

---

### PG-002 · Forgot Password

| Property | Value |

|------

**Layout:**

```
```

? ?

? ??

**States:** `form` ? `loading` ? `success` (always show success, even if email doesn't exist) | `error_rate_limit` (too many requests)

**?? API Gap:** Need to add:

```yar

POST /auth/reset-password

bod

---

### PG-003 · Reset Password

| Property | Value |

|------

**Layout:** Password input + confirm password + strength indicator. Token expiry: 1 hour.

**States:** `form` ? `loading` ? `success` ("Password changed. Redirecting to login...") | `error_expired` ("This link has expired. Request a new one.") | `error_weak` (inline validation)

---

### PG-004-007 · Onboarding Wizard (4 steps)

| Property | Value |

|------
```

| **Shell** | Auth |

| **Ac

**Step 1 - Organization Setup**

```

Organization Details

????

"You can change these later in Settings"

```

**Step 2 - First Facility:** Facility name*, address (street, city), phone, type (Clinic/Polyclinic/Hospital), opening hours (optional).

**Step 3 - Admin Account:** First name*, last name*, email*, password* (strength meter), confirm password.

**Step 4 - First Doctor (Skippable):** Name, specialty (dropdown), license number, default consultation duration (15/20/30 min).

**UX Rules:**

- ? In

- ? No partial org creation (single DB transaction)

**?? API Gaps:**

```yar

---

### PG-008 · Onboarding Complete

| Property | Value |

|------

**Layout:** Success animation (checkmark), org summary card, "Go to Dashboard" CTA. Auto-redirect after 5 seconds.

---

## 2. Global Layout & Navigation

### 2.1 Application Shell Architecture

MediCore uses **role-based shell routing**. Each shell has its own sidebar, but shares a common top bar.

```

????

**Shell assignments by role:**

| Role | Shell | Default Route | Sidebar Items |

|------

---

### PG-009 · Top Bar (Shared Component)

**Always visible**, sticky at top. Height: 56px (`var(--topbar-height)`).

```

????

**Components:**

| Elem
```

**?? API Gaps:**

```yar

---

### PG-010 · Facility Switcher (Shared Component)

**Always visible** in top bar or sidebar footer for multi-facility users.

```

????

**Behavior:**

- On

---

### PG-011-014 · Role-Aware Dashboards

#### PG-011 · Super Admin Dashboard (`/super/dashboard`) - **NEW page**

| Property | Value |

|------

| **API Calls** | `GET /super/tenants`, `GET /super/metrics` (**NEW**) |

**Layout:**

```

**?? This is an entirely new shell/page not in current docs.** Needed for SaaS management. Add to Phase 2.

---

#### PG-012 · Org Admin Dashboard (`/admin/dashboard`) - EXISTS ?

| Property | Value |

|------

**Layout (improved from audit):**

```

Row 2: Charts (2 columns)

????

Row 3: Facility Cards (if multi-facility)

????

Row 4: Recent Activity (full width)

? Tab

**UX Rules:**

- ? Da

---

#### PG-013 · Doctor Dashboard - **IMPROVED** (was missing as separate page)

Doctors currently land on `/clinical/patients`. The CodeCanyon audit reveals doctors need a proper **Today's Schedule** dashboard.

| Property | Value |

|------

**Layout:**

```

Row 2: Today's Schedule (main)

????

Row 3: Two panels

????
```

? Pending Lab Results  ? ? Recent Patients  ?

? (no

---

#### PG-014 · Receptionist Dashboard - **IMPROVED**

Receptionists currently land on `/ops/queue`. Better: a dedicated dashboard with quick actions.

| Property | Value |

|------

**Layout:**

```

Row 2: Two panels

????

Row 3: Alerts

? ?? 

---

## Audit Summary - Part 1

### API Gaps Found

| Missing Endpoint | Priority | Phase |

|------
```

| `GET /super/tenants` + `GET /super/metrics` | P2 | 2 |

**New Pages Found (vs current docs)**

| Page | Why Missing | Phase |
|---|---|---|

**Layout Improvements Over Current Docs**

| Page | Current | Improved |
|---|---|---|

---

> **Next:** Part 2 - Patient Management & Appointment Pages

---

# File: 04-pages-bible-part2.md

# Pages Bible - Part 2: Patient Management & Appointments

> Cross-referenced against: `openapi.yaml`, `04-screen-inventory.md`, `02-SRS.md`

---

## 3. Patient Management Pages

**PG-020 · Patient List**

| Property | Value |
|---|---|

| **Access** | RECEPTIONIST, DOCTOR, NURSE, FACILITY_ADMIN |

| **AP

**Layout:**

```

**States:** `loading` (skeleton table) ? `populated` ? `empty` ("No patients yet. Register your first ?") ? `search_results` ? `no_results`

**UX Rules:**

- ? De

**API alignment:** `GET /patients/search` exists ?. Needs filter params:

```yar

## ENHANCED - add these query params

GET
/patie
limit=

---

**PG-021 · New Patient / Edit Patient**

| Property | Value |
|---|---|

**Layout:**

```
```

? ?

????

**States:** `form` ? `validating` ? `saving` ? `success` (show MRN in success card) | `error_duplicate_phone` (?? warning, not blocker)

**UX Rules:**

- ? M

**API Gaps:**

```yar

---

### PG-022 · Patient Profile & Timeline

| Property | Value |

|------

**Layout:**

```

? ? Ahmed Benali ? MRN: CEA-2026-00001 ? 42M ?

? ? +

**UX Rules:**

- ? Pa

- ? Timeline is chronological, combines encounters + labs + radiology + invoices + messages

- ? "L

**API Gaps:**

```yar

---

## 4. Appointments & Scheduling Pages

### PG-030 · Doctor Schedule Management

| Property | Value |

|------

**Layout:**

```

? ????????????????????????????????????????????????????????? ?

? Cli

**UX Rules:**

- ? Sp

**API Gaps:**

```yar

---

### PG-031 · Appointment Calendar

| Property | Value |

|------

**Layout:**

```

? Calendar Grid (Day view shown) ?

? 08

**States:** `loading` ? `day_view` | `week_view` | `doctor_column_view` | `room_view`

**UX Rules:**

- ? Cl

---

### PG-032 · Appointment List View

| Property | Value |

|------

**Layout:**

```
```

? Time? Patient ? Doctor ? Type ? Status ?Actions ?

????

**API Gaps:**

```yar

---

**PG-033 · Booking Form (Modal/Drawer)**

| Property | Value |

|------

**Layout:**

```

? Type: [Consultation ?] ?

? Re

**UX Rules:**

- ? If

---

**PG-034 · Check-in & Queue Management**

| Property | Value |

|------

**Layout:**

```
```

? ? [+ Register Walk-in] ? ? Waiting times: ?

? ??

**UX Rules:**

- ? Cl

---

**PG-035 · Queue Display (Public TV)**

| Property | Value |

|------

**Layout:**

```
```

? ? Average wait: 15 minutes ?

????

---

## Audit Summary - Part 2

### API Gaps Found

| Missing Endpoint | Priority | Phase |

|------

### New Pages Found

| Page | Route | Phase |

|------

### Layout Improvements

| Page | Improvement |

|------

| Queue | Added walk-in support, waiting time highlights, live connection indicator |

---

> **Next:** Part 3 - Encounters/EMR & Billing Pages

---

# File: 04-pages-bible-part3.md

# Pages Bible - Part 3: Encounters/EMR & Billing

> Cross-referenced against: `openapi.yaml`, `04-screen-inventory.md`, `02-SRS.md`, `DESIGN.md`

---

## 5. Encounter / EMR Pages

### PG-040 · Encounter Workspace (Doctor View) - **Hero Page**

| Property | Value |
|---|---|

**3-Column Layout:**

```
```

? ?? Problems ??? ??? SOAP Notes ??????????????  ? ? Differentials?

? E11

**Sub-component States:**

| Component | States |

|------

| Prescription | `empty` ? `drug_search` ? `form` ? `allergy_warning` ? `added` |

| Lab

**UX Rules:**

- ? Au

**API alignment:** All encounter APIs exist ?. Gaps:

```yar

---

### PG-041 · Vitals Panel (sub-component or dedicated tab)

Already part of encounter workspace (PG-040 center column, top section).

**Additional detail:**

- Fiel

---

### PG-042 · Prescription Writer (sub-component)
```

Part of encounter workspace (PG-040 center column).

**Drug entry flow:**

```

**UX Rules:**

- ? Q

---

### PG-043 · Encounter Sign-off (Modal)

| Property | Value |

|------
```

**Layout:**

```

**Post sign-off events:**

1. `en

---

## 6. Billing & Finance Pages

### PG-050 · Invoice List

| Property | Value |

|------
```

| **API Calls** | `GET /invoices?status=&facility_id=&from_date=&to_date=&doctor_id=&page=` |

**Layout:**

```

**UX Rules:**

- ? Ru

---

### PG-051 · Invoice Detail / Edit

| Property | Value |

|------

**Layout:**

```

? LINE ITEMS ?

? ??

**States:** `draft` (editable) ? `sent` ? `partially_paid` ? `paid` | `overdue` | `void`

**UX Rules:**

- ? Li

**API Gaps:**

```yar
```

## PG-052 · Record Payment (Modal)

| Property | Value |

|------

**Layout:**

```

---

## PG-053 · Revenue Dashboard

| Property | Value |

|------
```

| **Route** | `/admin/analytics` (finance tab) |

| **AP

**Layout:**

```

**API Gaps:**

```yar

---

## Audit Summary - Part 3

### API Gaps Found

| Missing Endpoint | Priority | Phase |

|------

| `POST /encounters/{id}/lab-orders` | P0 | 1 |

| `PO

**Layout Improvements**

| Page | Improvement |

|------

---

> **Next:** Part 4 - Pharmacy, Lab, Radiology & Inventory Pages

---

# File: 04-pages-bible-part4.md

# Pages Bible - Part 4: Pharmacy, Lab, Radiology & Inventory

> Phase 2 modules. Cross-referenced against: `openapi-phase2.yaml`, `04-screens-phase2-3.md`

---

## 7. Pharmacy Module (Phase 2)

**PG-060 · Dispensing Queue**

| Property | Value |

|------

| **Route** | `/pharmacy/queue` |

| **Sh

**Layout:**

```

**UX Rules:**

- ? Li

---

### PG-061 · Dispense Detail

| Property | Value |

|------

**Layout:**

```

? PRESCRIPTION ?

? ??

**UX Rules:**

- ? Au

---

### PG-062 · Drug Catalog

| Route | `/pharmacy/drugs` |

|------

**Layout:** Data table - columns: Name, Generic, Form (tablet/syrup/...), Strength, Category, Company, Unit, Price, Stock. CRUD modal for add/edit. Search + filter by category.

**CodeCanyon alignment:** Smart Hospital has explicit `Medicine Category`, `Medicine Company`, `Medicine Unit` sub-pages. MediCore simplifies: categories/companies managed as dropdown values within drug

create/edit modal (**not separate pages**).

---

### PG-063 · Pharmacy Inventory

| Route | `/pharmacy/inventory` |
|---|

**Layout:**

```
Table: Drug ? Batch ? Expiry ? In Stock ? Reorder Level ? Status
Actions: [Adjust Stock] [Create PO] [Export]
```

---

### PG-064-065 · Purchase Orders & Suppliers

Standard CRUD pages for PO lifecycle (create ? approve ? receive ? close) and supplier management. See `04-screens-phase2-3.md` S74-S75 for existing spec. No major gaps.

---

## 8. Lab (Pathology) Module (Phase 2)

### PG-070 · Lab Orders Queue

| Property | Value |
|---|

**Layout:**

```

????

---

### PG-071 · Lab Order Detail & Result Entry

| Property | Value |

|------

**Layout:**

```

? ? Hgb ? [14.2] ? 13.5-17.5 ? g/dL ?? NORMAL ??

? ? C

**UX Rules:**

- ? Au

---

### PG-072 · Lab Test Catalog

| Route | `/lab/catalog` |

|------

Standard CRUD: Test name, code, specimen type (blood/urine/stool), normal ranges (by age/sex), department, price. Panel grouping (e.g., "CBC Panel" = WBC + RBC + Hgb + Plt).

---

## 9. Radiology Module (Phase 2)

### PG-080 · Radiology Queue

Same pattern as lab queue. Route: `/radiology/queue`. Columns: Order#, Patient, Modality (X-ray/US/CT/MRI), Body Part, Status.

### PG-081 · Radiology Report Entry

| Route | `/radiology/orders/[orderId]` |
|---|

**Layout:**

```

**Key difference from lab:** Text-based report (findings + impression) instead of structured numeric results. Template picker for common study types. Image upload to R2.

### PG-082 · DICOM Viewer

Route: `/radiology/viewer/[studyId]`. Embeds Cornerstone.js viewer or Orthanc iframe. Full-screen, dark background. Measurement tools, window/level adjustment, zoom.

---

## 10. Inventory Module (Phase 2)

### PG-090 · Inventory Dashboard

| Route | `/ops/inventory` |
|---|
```

|-------|-------------------|

**Layout:**

```

KPI: [Total items: 342] [Total value: 28.4K MAD] [Categories: 18]

Table: Item ? Category ? Unit ? In Stock ? Reorder ? Status ? Actions

????

### PG-091-094 · Item Catalog, Stock Transactions, Purchase Orders, Suppliers

Standard CRUD pages matching `04-screens-phase2-3.md` S100-S104. Key features:

- **Ite

---

## Audit Summary - Part 4

### API Gaps (Phase 2)

| Missing Endpoint | Already in openapi-phase2.yaml? |

|------

### New UX Patterns Found
```

| Pattern | Description |

|------

---

> **Next:** Part 5 - IPD, HR, Portal, Settings & Audit Pages

---

# File: 04-pages-bible-part5.md

# Pages Bible - Part 5: IPD, HR, Portal, Settings & Audit

> Final part. Cross-referenced against: `openapi-phase2.yaml`, `openapi-phase3.yaml`, `04-screens-phase2-3.md`

---

## 11. IPD (Inpatient) Module (Phase 2)

### PG-100 · Active Admissions

| Property | Value |

|------

**Layout:**

```
```

? ADM-10 ? Omar K.  ? WardB? B-02 ? Dr. Y  ? 14 Apr ? 1  ?

????

---

### PG-101 · Admission Detail

| Route | `/ipd/admissions/[admissionId]` |

|------

**Layout:**

```
```

? ????????????????????????????????????????????????????????????

? ?T

---

**PG-102 · Bed Map**

| Route | `/ipd/beds` |

|------

**Layout:**

```

ICU (4 beds - 75% occupancy)

????

Ward A (8 beds - 50% occupancy)

????

Legend: ? Available  ? Occupied  ? Maintenance

Click
```

```

**Responsive:** Grid on desktop, list view on mobile (<768px).

---

### PG-103 · Nursing Station

| Route | `/ipd/nursing-station` |
|---|

**Layout:** Per-patient task cards for the shift. Each card shows: patient name, bed, pending tasks (vitals due, medications to administer, doctor orders). Tasks sorted by urgency/time. Checkboxes to mark completed. See Part 1 user journey (Nurse Khadija) for full flow.

### PG-104 · Discharge

| Route | `/ipd/discharge/[admissionId]` |
|---|

**Layout:** Discharge summary form: discharge diagnosis, condition at discharge, follow-up instructions, medication on discharge, diet instructions. PDF generation for patient. Triggers: final invoice, bed freed, patient notified.

---

## 12. HR Module (Phase 3)

### PG-110 · Staff Directory

| Route | `/hr/staff` |
|---|

Table: Name, Role, Department, Facility, Status, Phone, Email. Filters: role, department, facility, status. Click ? Staff Detail (profile, schedule, HR docs, attendance history).

### PG-111 · Attendance Dashboard

| Route | `/hr/attendance` |
|---|

Calendar heatmap per staff member. Check-in/out tracking. Late arrivals highlighted. Monthly summary cards: present, absent, late, leave.
```

### PG-112 · Leave Management

| Route | `/hr/leave` |

|------

Leave requests table (staff, type, dates, status). Approval workflow: submit ? manager reviews ? approve/reject. Calendar overlay showing who's on leave.

### PG-113 · Payroll

| Route | `/hr/payroll` |

|------

Monthly payroll generator: select month ? calculate (base + overtime - deductions). Payslip PDF per staff member. Batch approval and payment tracking.

---

## 13. Patient Portal (Phase 2)

### PG-120 · Portal Login

| Route | `/portal/login` |

|------

**Layout:**

```
```

No password - OTP only for patients. Simple, mobile-optimized.

---

### PG-121 · Portal Dashboard

| Route | `/portal/dashboard` |

|------

**Layout:**

```

???????????????? ????????????????????? ?????????????????????

? ? N

Quick Actions: [Book Appointment] [View Records] [Pay Invoice]

Recent Activity:

? ? A

**UX:** Drastically simpler than staff views. No jargon. Large touch targets. Mobile-first.

---

### PG-122-125 · My Records, My Appointments, My Invoices, Settings

Already specified in `04-screens-phase2-3.md` S52-S55. Key additions from CodeCanyon audit:

- **My

---

## 14. Settings & Admin Pages

### PG-130 · Organization Settings
```

| Route | `/admin/settings` |

**Sub-pages/tabs:**

| Tab | Contents |

**UX Rules:**

---

### PG-131 · Team / User Management

| Route | `/admin/team` |

**Layout:**

```

[+ Invite Member] ? Modal:

Click user ? User Detail:
```

"Effective permissions" preview, suspend/reactivate actions

```

**API Gaps:**

```yaml

---

### PG-132 · Audit Log

| Route | `/admin/audit-log` |

|------

**Layout:**

```

Filters: [User ?] [Entity type ?] [Action ?] [Date range] [Facility ?]

Click

---

### PG-133 · Reports Center

| Route | `/admin/reports` |

|------

**Layout:**

```

Clinical: [Patient Summary] [Encounter Stats] [Diagnosis Frequency]

Operational: [Appointment Analysis] [Queue Wait Times] [No-show Rate]

Finan

Each report: Filter panel + Table/Chart + [Export CSV] [Export PDF]

Save

---

## 15. Cross-Cutting: Page Relations Map

```mermaid

graph

subgraph "Appointment Flow"

subgraph "Post-Encounter"

subgraph "Pharmacy Flow"

subgraph "Lab Flow"
```

```
    subgraph "IPD Flow"

    subgraph "Patient Portal"

    INV --> REV[PG-053 Revenue Dashboard]
```

---

## 16. Complete Page Count (Final)

| Category | Pages | Phase |
|---|---|---|

---

## 17. Master API Gap Summary (All Parts)

### Phase 1 (MVP) - Must Add

| Endpoint | Used By |

|------

### Phase 2 - Must Add

| Endpoint | Used By |

|------

**Total new endpoints identified: 25** (15 MVP, 10 Phase 2)

---

> All 5 parts complete. The Pages Bible is ready at:

> - `0

# File: 04-screen-inventory.md

# Phase 4 - Screen Inventory

## MediCore HIS - Complete Screen List with States

---

### Auth Screens

| # | Screen | Route | States | Key Components | Linked From |
|---|---|---|---|---|---|

### Admin Screens

| # | Screen | Route | States | Key Components | Linked From |
|---|---|---|---|---|---|

| S15 | Audit Log | `/admin/audit-log` | loading, populated, empty, filtered | Audit table (time, user, action, entity, details), filter dropdowns | Sidebar nav |

| S16
forma

## Clinical Screens

| # | Screen | Route | States | Key Components | Linked From |

|---|---

### Encounter Editor Sub-States

| Component | States |

|------

## Operations Screens

| # | Screen | Route | States | Key Components | Linked From |

|---|---

| S35 | Billing List | `/ops/billing` | loading, populated, empty, filtered | Invoice table (number, patient, amount, status, date), status filter, date range filter | Sidebar nav |

| S36
items
butto

## Public Screens

| # | Screen | Route | States | Key Components | Linked From |

|---|---

---

## Shared Components

| Component | Used In | Description |

|------

---

## Screen Count Summary

| Category | Count |

|------

---

# File: 04-screens-phase2-3.md

# Phase 2 & 3 - Information Architecture & Screen Inventory

## MediCore HIS - All Routes, Screens & Navigation (Full Product)

> Extends `04-information-architecture.md` and `04-screen-inventory.md`.

---

## 1. Full Site Map (All Phases)

```mermaid

graph

    ROOT --> AUTH["/(auth)"]

RC

    ADMIN --> A1["/admin/dashboard"]

AD
```

```
ADMIN --> A4["/admin/services"]

CLINICAL --> C1["/clinical/patients"]

OPS --> O1["/ops/patients"]

LAB_SHELL --> L1["/lab/queue"]

PHARMA_SHELL --> PH1["/pharmacy/queue"]

RAD_SHELL --> R1["/radiology/queue"]

IPD_SHELL --> I1["/ipd/admissions"]
```

```
    IPD_SHELL --> I2["/ipd/admissions/[admissionId]"]

    HR_SHELL --> H1["/hr/staff"]

    PORTAL_SHELL --> PT1["/portal/login"]

    CMS_PUBLIC --> CM1["/[orgSlug] - public clinic page"]

    PUBLIC --> PUB1["/queue-display/[facilityId]"]
```

---

## 2. New Navigation Sidebars (Phase 2/3)

### Lab Sidebar (LAB_TECH, DOCTOR)

```

### Pharmacy Sidebar (PHARMACIST)

```

? Drug Catalog

? Pur

**Radiology Sidebar (RADIOLOGIST)**

```

**IPD Sidebar (DOCTOR, NURSE, FACILITY_ADMIN)**

```

**HR Sidebar (HR_MANAGER, ORG_ADMIN)**

```

**Portal Sidebar (PATIENT - patient portal)**

```

**Admin Sidebar Expansion (P2 additions)**

```

? Das

---

## 3. Phase 2 Screen Inventory

### Patient Portal (6 screens)

| # | Screen | Route | States | Key Components |

|---|---

### Lab Module (4 screens)

| # | Screen | Route | States | Key Components |
```

|---|--------|-------|--------|----------------|

| S60
statu

### Pharmacy Module (6 screens)

| # | Screen | Route | States | Key Components |

|---|--

### Telemedicine (3 screens)

| # | Screen | Route | States | Key Components |

|---|--

### IPD Module (7 screens)

| # | Screen | Route | States | Key Components |

|---|--

| S91 | Admission Detail | `/ipd/admissions/[admissionId]` | loading, active, discharged | Patient summary, bed info, round notes timeline, medication schedule, vitals chart, "Transfer Bed" and "Discharge" |

| S92
count

### Inventory Module (5 screens)

| # | Screen | Route | States | Key Components |

|---|---

### Radiology Module (4 screens)

| # | Screen | Route | States | Key Components |

|---|---

### Messaging (3 screens)

| # | Screen | Route | States | Key Components |

|---|--------|-------|--------|----------------|

| S12
(sidel

### Admin Expansion (P2)

| # | Screen | Route | States | Key Components |

|---|---

### Other Public/Widget Screens (P2)

| # | Screen | Route | States | Key Components |

|---|---

---

## Phase 3 Screens (HR + Workflow + Analytics)

| # | Screen | Route | Screens |

|---|---

| S160-S163 | Workflow Engine | `/admin/workflows/*` | Workflow list, Rule builder (simple), Visual canvas builder, Execution log |

| S16

---

## 4. Complete Screen Count

| Category | Phase 1 | Phase 2 | Phase 3 | Total |

|------

---

# File: 04-user-journey-maps.md

# Phase 4 - User Journey Maps

## MediCore HIS - Per-Persona User Journeys

---

### Journey 1: Dr. Youssef - Solo Practitioner First Day

```mermaid

flowc
```

```
B --> C{"Onboarding Wizard<br/>(4 steps)"}

H --> I["Sets weekly schedule<br/>Mon-Fri 9:00-13:00, 15:00-18:00<br/>15-min slots"]

J --> K["? First patient calls"]

W --> X{"End of day"}

style Q fill:#4CAF50,color:#fff
```

### Success State

### Failure States

## Journey 2: Fatima (Receptionist) - Typical Morning

```mermaid
flowc

    B --> C["? Walk-in patient arrives"]
C -

    E --> H["Books appointment<br/>Selects Dr. Youssef ? Today ? 10:30"]
H -

    I --> J["? 10:25 - Patient called"]
J -

    M --> N["Doctor takes patient"]
N -

    P --> Q["? Invoice appears in<br/>/ops/billing"]
Q -

    T --> V["Receipt printed/emailed"]
V -

    style K fill:#4CAF50,color:#fff
sty
```

### Success State

Fatim
consu

### Failure States

- **Double-booking**: Two patients booked at 10:30. **Mitigation**: Redis optimistic lock prevents this; slot greys out immediately after booking.

---

## Journey 3: Dr. Amina - Multi-Facility Admin Review

```mermaid

    B --> C["Switches to Facility A<br/>(Rabat flagship)"]

    D --> E["Switches to Facility B<br/>(Salé branch)"]

    F --> G["Opens /admin/analytics"]

    H --> I["Opens /admin/team"]

    J --> K["Opens /admin/audit-log"]

    L --> M["Satisfied with data segregation"]

    style N fill:#2196F3,color:#fff
```

### Success State

### Failure States

- **Analytics are wrong**: Revenue doesn't match manual calculations. **Mitigation**: Revenue aggregates pulled from PG invoices (source of truth), not calculated client-side.

---

# File: 04-user-journey-maps-phase2-3.md

# Phase 2 & 3 - User Journey Maps

## MediCore HIS - Extended Journey Maps for New Personas

---

## 1. Patient (Fatima) - Portal Self-Service Journey

```mermaid
flowc

E --> F["? Dashboard"]
F -

G -->|View records| H["? My Records"]
H -

G -->|Book appointment| L["? Book Appointment"]
L -

G -->|Pay bill| R["? My Invoices"]
R -
```

G -->|Cancel appointment| X["? My Appointments"]

X -

style E fill:#4CAF50,color:#fff

sty

**Touchpoints**: SMS, Web Portal, Stripe

**Pai

---

## 2. Lab Technician (Hassan) - Lab Order to Result Journey

```mermaid

flowc

C --> D["Collects blood sample"]

D -

F --> G["Runs tests on analyzer"]

G -

I --> J["Clicks 'Submit for Authorization'"]

J -

style I fill:#FF9800,color:#fff

sty

**Touchpoints**: Lab queue dashboard, specimen scanner, result entry form

**Pai
```

**Emotions**: Efficient (structured workflow), Confident (abnormal flags catch errors)

---

## 3. Pharmacist (Nadia) - Dispensing Journey

```mermaid
flowc

    B --> C["?? ALERT: Patient allergic to Penicillin<br/>Amoxicillin is a penicillin derivative!"]
C -

    D -->|Override| H["Enter override reason"]

    G --> I["Checks stock: Azithromycin 250mg<br/>Stock: 120 tablets, Batch: AZ-2026-003"]
I --

    K --> O{"Stock below reorder?"}
O -

    style C fill:#F44336,color:#fff
sty
```

**Touchpoints**: Pharmacy queue, allergy checker, stock manager, messaging

**Pai

---

## 4. Nurse (Khadija) - IPD Nursing Station Journey

```mermaid
flowc
```

B --> C["Patient card: Mr. Ben Ali<br/>Room: Ward A - Bed 3"]

D --> E["? Vitals at 07:00 ?<br/>BP: 130/82, HR: 78, Temp: 37.0"]

E --> I["Records vitals in form<br/>? Saved to admission record"]

F --> J["08:00: Administers medication"]

K & L --> M["All morning tasks complete ?"]

H --> O["Adjusts IV drip rate<br/>Notes: 'Rate increased per Dr. orders'"]

I --> P["? Vitals chart updates<br/>BP trending downward (good)"]

style M fill:#4CAF50,color:#fff

**Touchpoints**: Nursing station dashboard, vitals entry, medication log

---

## 5. Admin (Karim) - Workflow Automation Journey (Phase 3)

```mermaid

D --> E["Trigger: encounter.signed_off"]

E & F & G --> H["Karim clicks 'Test with sample'"]
```

H --> I["Dry run result:<br/>'Conditions MET - SMS would send to patient phone'"]

I --> J["Karim activates workflow"]

J --

K --> L["A week later:<br/>Dr. Youssef signs off encounter<br/>Diagnosis: E11.2 (Type 2 Diabetes)"]

L --

N --> O["Karim checks workflow log:<br/>12 executions, 100% success"]

style I fill:#FF9800,color:#fff

sty

**Touchpoints**: Workflow builder, template library, execution log

**Pai

---

## 6. Doctor (Dr. Youssef) - Telemedicine Journey

```mermaid

flowc

C --> E["Patient opens link ? Waiting room<br/>'Doctor will join shortly'"]

D --> F["Dr. Youssef clicks link at 14:00"]

F --

G --> H["Split-screen view:<br/>? Video on left<br/>? Encounter form on right"]

H --> I["Records vitals (patient reports)<br/>Types SOAP notes<br/>Clicks AI copilot ? SOAP expanded"]

I --

K --> L["Duration: 18 minutes"]

L --

style G fill:#4CAF50,color:#fff
```

style O fill:#4CAF50,color:#fff

```

**Touchpoints**: Video room, encounter sidebar, portal billing

**Pai

---

# File: 05-database-schema.md

# Phase 5 - Database Schema

## MediCore HIS - Full Schema Documentation

> The complete Prisma schema lives at `his-predev/schemas/postgres/schema.prisma` (376 lines).

> Mo

---

## 1. PostgreSQL Schema (Prisma ORM)

### 1.1 Tenancy Layer

#### `organizations`

| Colu
```

**Design rationale**: `mrn_counter` lives in PG (not Mongo) because it needs atomic `$transaction` increment to prevent duplicate MRNs under concurrent registration.

#### `facilities`

| Colu

#### `departments`

| Colu

**Phase note**: Departments are a stub in MVP. The table exists for the data model but no CRUD UI is built until Hospital tier (Phase 3).

---

### 1.2 Auth Layer

#### `users`

| Colu

| `doctor_license_no` | `String?` | | Medical license number |

| `doc

**Index**: `(org_id, status)` - fast lookup of active users within an org.

**Design rationale**: Doctor-specific fields live on User (not a separate table) because a User IS a Doctor in this context. Separate tables would require JOINs on every encounter query. Null fields for non-doctors are acceptable at this scale.

#### `user_facility_roles`

| Colu

**Unique**: `(user_id, facility_id)` - one role per user per facility.

**Inde

#### `refresh_tokens`

| Colu

**Design rationale**: We store the hash, not the plaintext token. The actual token is only in the HttpOnly cookie. This way, a database breach doesn't give attackers valid tokens.

---

### 1.3 Billing Layer

#### `services` (Services Catalog)

| Colu

| `name` | `String` | NOT NULL | "General Consultation" |

| `des

#### `invoices`

| Colu

**Indexes**: `(org_id, facility_id, status)`, `(org_id, patient_id)`.

#### `payments`

| Colu

---

### 1.4 Audit Layer

#### `audit_logs`

| Colu

**Indexes**: `(org_id, entity_type, entity_id)`, `(org_id, user_id)`.

**Design rationale**: Audit logs are append-only. No UPDATE or DELETE operations ever. This is critical for compliance and forensic analysis.

---

## 2. MongoDB Schema (Mongoose ODM)

### 2.1 `patients` Collection

**Full JSON schema**: `his-predev/schemas/mongo/patient.schema.json` (104 lines)

**Key design decisions**:

- `org

**Indexes**:

- `{ or

### 2.2 `encounters` Collection

**Full JSON schema**: `his-predev/schemas/mongo/encounter.schema.json` (129 lines)

**Key design decisions**:

- **Or

**Indexes**:

- `{ or

### 2.3 `appointments` Collection

**Stored in MongoDB** (not defined in his-predev JSON schema but modeled in his-predev architecture):

| Field | Type | Notes |

|------

**Indexes**:

- `{ or

### 2.4 `notification_logs` Collection

| Field | Type | Notes |

|------

---

### 3. Redis Key Patterns

| Pattern | TTL | Type | Purpose |

|------

---

# File: 05-data-flow.md

# Phase 5 - Data Flow

### MediCore HIS - End-to-End Data Flow Diagrams

---

## 1. Patient Registration Flow

```mermaid
flowc

    style H fill:#FF9800,color:#fff
sty
```

**Cross-DB note**: MRN counter is in PostgreSQL (atomic increment), patient document is in MongoDB. If Mongo write fails after PG counter increments, we accept a gap in MRN sequence (next registration gets counter+1). This is acceptable - MRN gaps are common in medical systems.

---

## 2. Appointment Booking Flow

```mermaid
flowc
```

J -->|No| K["409 Conflict:<br/>Slot already booked"]

J -

style F fill:#FF9800,color:#fff

sty

**Double-booking prevention**: The Redis lock (30-second TTL, NX flag) ensures that even under concurrent requests, only one booking for a given doctor+slot can proceed at a time. The lock is deleted immediately after the Mongo write succeeds. If the process crashes mid-booking, the lock auto-expires after 30 seconds.

---

## 3. Encounter Sign-Off Flow (Most Complex)

```mermaid

flowc

F --> G["Billing Subscriber<br/>(async)"]

F -

G --> G1["Fetch services catalog<br/>(PG: services)"]

G1

I -->|Yes| J["Redis: Publish<br/>clinics.prescription.issued"]

I --

J --> J1["(Phase 2: Pharmacy subscriber)"]

H --> H1["Log notification_logs<br/>(Mongo)"]

E --> L["200 OK<br/>{encounter, invoice_id}"]
```

style E fill:#4CAF50,color:#fff

sty

**Cross-DB consistency**: The sign-off creates data in both MongoDB (encounter status) and PostgreSQL (invoice). The outbox pattern ensures:

1. En

---

## 4. AI Copilot Request Flow

```mermaid

flowc

style Q fill:#9C27B0,color:#fff

sty
```

```

**Key design rules**:

- Fas

---

## 5. SSE Queue Update Flow

```mermaid

flowc

G["Queue Display TV<br/>EventSource connection"] --> H["Receives SSE event"]

H -

J["Ops Queue Screen<br/>EventSource connection"] --> K["Receives SSE event"]

K -

style C fill:#FF9800,color:#fff

sty

**SSE architecture**: NestJS uses `@Sse()` decorator with Redis subscriber. Each connected client specifies their `facilityId`. The SSE controller filters events and only pushes to clients watching the relevant facility. The public queue display (`/queue-display/:facilityId`) uses the same SSE stream but requires no authentication.

---

## 6. End-to-End Data Flow Summary

```mermaid

flowc
```

```
    UI["Browser<br/>(Next.js)"]

subgraph "API Layer"

subgraph "Event Layer"

subgraph "Storage"

subgraph "External"

UI -->|REST + SSE| NEST
```

---

## File: 05-data-flow-phase2-3.md

# Phase 2 & 3 - Data Flow Diagrams

## MediCore HIS - Key Data Flows for Extended Modules

---

## 1. Patient Portal - OTP Login Flow

```mermaid

flowc

style C fill:#FF9800,color:#fff

sty

---

## 2. Lab Order ? Result ? Billing Flow

```mermaid

flowc

F --> G["Lab tech: Specimen collection<br/>PATCH /lab/orders/:id/specimen<br/>{specimen_id: 'SPL-2026-00042'}"]

G --> H["MongoDB: Update LabOrder<br/>status: specimen_collected"]

H --> I["Lab tech: Enter results<br/>PUT /lab/orders/:id/results"]

I --

K --> L["Supervisor: Authorize<br/>POST /lab/orders/:id/authorize"]

L -

N --> O["Billing subscriber:<br/>Auto-add lab charges to invoice"]

N -

style D fill:#4CAF50,color:#fff

sty

---

## 3. Pharmacy - Prescription to Dispensing Flow

```mermaid

flowc

C --> D["Pharmacist opens /pharmacy/queue<br/>Sees pending Rx"]

D -
```

R -->|Yes| S["Redis: Publish pharmacy.stock.low"]

S -

style N fill:#4CAF50,color:#fff

sty

---

## 4. IPD - Admission to Discharge Flow

```mermaid

flowc

H --> I["Daily cron at midnight"]

I --

F --> L["Nursing station sees patient"]

L -

F --> O["Doctor daily rounds<br/>POST /ipd/admissions/:id/round-notes"]

O -

P --> Q["Doctor decides: Discharge<br/>POST /ipd/admissions/:id/discharge"]

Q -

style F fill:#4CAF50,color:#fff

sty

---
```

## 5. Telemedicine - Video Consultation Flow

```mermaid
flowc

D --> E["Patient opens /portal/telemedicine/:sessionId"]
E -

D --> G["Doctor opens /clinical/telemedicine/:sessionId"]
G -

J --> K["Doctor has encounter sidebar<br/>Can record vitals, SOAP, diagnosis, Rx"]
K -

style C fill:#4CAF50,color:#fff
sty
```

---

## 6. Triage Chatbot Flow

```mermaid
flowc
```

I -->|Green| L["? ROUTINE: 'Book at your convenience'<br/>? Booking flow"]

K -

style H fill:#9C27B0,color:#fff

```

---

## 7. Workflow Automation Flow (Phase 3)

```mermaid

flowc

style I fill:#4CAF50,color:#fff

```

---

## 8. Ambient Audio AI Flow (Phase 2)

```mermaid

flowc

F --> G["AI: Process transcription<br/>Extract structured SOAP"]

G -

A --> M["Audio file saved to R2<br/>(if recording consent = true)"]

M -

style E fill:#9C27B0,color:#fff

sty

---

## 9. Internal Messaging Flow (Phase 2)

```mermaid

flowc

C --> D["User clicks conversation or 'New'"]

D -

F --> G["User types message ? Send"]

G -

H --> L["Redis: Publish messaging.new_message"]

L -

style I fill:#4CAF50,color:#fff

sty

---

# File: 05-ERD.md

# Phase 5 - Entity Relationship Diagram
```

## PostgreSQL ERD

```mermaid
erDia

    Facility ||--o{ UserFacilityRole : "assigns"
Fa

    User ||--o{ UserFacilityRole : "has roles"
Us

    Invoice ||--o{ InvoiceItem : "contains"
Inv

    Organization {
    Facility {
```

string org_id FK

User {

UserFacilityRole {

RefreshToken {

DoctorSchedule {

```
    int day_of_week "0-6"
}

Service {
}

TaxConfig {
}

Invoice {
}
```

InvoiceItem {

Payment {

NotificationTemplate {

AuditLog {

```

---

## MongoDB ERD

```mermaid
erDia

    Patient {

    Appointment {
```
```

```
    string notes
}

Encounter {
```

---

### Cross-Database References

| Source (DB) | Field | Target (DB) | Field | Type |
|---|---|---|---|---|

> **Note**: Cross-database references are **string-based, not enforced by a foreign key**. Application logic in the repository layer is responsible for ensuring referential integrity. The outbox pattern (see `02-database-strategy.md`) handles the critical Encounter ? Invoice cross-DB write.

---

## File: 05-ERD-phase2-3.md

# Phase 2 & 3 - ERD Expansion

## MediCore HIS - Full Entity Relationship Diagram (All Phases)

> Extends `05-ERD.md`. MVP entities remain unchanged. This adds ~25 new entities.

---

## New PostgreSQL Entities (Phase 2)

```mermaid
erDia

    Facility ||--o{ Ward : "houses"
Fa

    Ward ||--o{ Bed : "contains"

    Drug ||--o{ PharmacyInventory : "stocked as"

    Supplier ||--o{ PurchaseOrder : "receives"
Pu

    LabTestCatalog {
S

    Drug {
```

string id PK "cuid()"

PharmacyInventory {

Supplier {

PurchaseOrder {

datetime approved_at

PurchaseOrderItem {

Ward {

Bed {

InventoryItem {

StockTransaction {

OrgModule {

CustomFieldDef {

Translation {

---

## New PostgreSQL Entities (Phase 3)

```mermaid
```

```
erDiagram
    User ||--|| StaffProfile : "has"
    Workflow ||--o{ WorkflowExecution : "runs"
    StaffProfile {
    AttendanceRecord {
    LeaveRequest {
```

string reason

PayrollRecord {

Workflow {

WorkflowExecution {

Ambulance {

string type "basic, advanced, neonatal"

AmbulanceDispatch {

---

## New MongoDB Collections (Phase 2)

```mermaid
erDia

LabOrder {
```

string invoice_id "PG Invoice.id"

}

RadiologyOrder {

Admission {

VideoSession {

ObjectId org_id

DispensingLog {

Message {

Conversation {

ObjectId _id PK

TriageSession {

CmsPage {

CampaignLog {

datetime scheduled_at

---

### Cross-Database References (Phase 2 Additions)

| Source (DB) | Field | Target (DB) | Field | Direction |
|---|---|---|---|---|

---

# File: 06-privacy-compliance.md

# Phase 6 - Privacy & Compliance

### MediCore HIS - GDPR-Aligned Privacy Framework

> **Compliance posture**: Morocco-first, GDPR-style data protection from day one. HIPAA-ready architecture designed in but not certified until US expansion. No formal certification required to launch in Morocco in Year 1.

---

## 1. Data Inventory - What We Collect and Why

#### 1.1 Personal Data

| Data Category | Specific Fields | Legal Basis | Retention | Storage |
|---|---|---|---|---|

### 1.2 Special Category Data (Sensitive)

| Data | Classification | Extra Protections |
|------

---

## 2. GDPR Rights Implementation

### 2.1 Right to Access (Art. 15)

| Requirement | Implementation |
|------

**MVP approach**: Admin manually triggers export via dashboard. Automated self-service in Phase 2 (patient portal).

### 2.2 Right to Rectification (Art. 16)

| Requirement | Implementation |

|------

### 2.3 Right to Erasure / Right to be Forgotten (Art. 17)

| Requirement | Implementation |

|------

**Anonymization strategy**:

- `nar

### 2.4 Right to Data Portability (Art. 20)

| Requirement | Implementation |

|------

### 2.5 Right to Object (Art. 21)

| Requirement | Implementation |

|------

## 3. Consent Management

### 3.1 MVP Consent Model

| Consent | Type | Collection Point | Storage |
|---|---|---|---|

### 3.2 Phase 2 Consent Model (Patient Portal)

| Consent | Type | Collection Point |
|---|---|---|

---

## 4. Data Retention Policy

| Data Type | Active Retention | Archive Retention | Deletion |
|---|---|---|---|

**Retention enforcement**: Monthly cron job checks retention dates and triggers anonymization/deletion. Logged in audit trail.

---

## 5. Data Protection Impact Assessment (DPIA) - Summary

### 5.1 Processing Activities Requiring DPIA

| Activity | Risk Level | Justification |
|---|---|---|

### 5.2 Mitigations for AI Data Processing

| Risk | Mitigation |
|---|---|

### 5.3 Mitigations for Multi-Tenancy

| Risk | Mitigation |
|---|---|

---

## 6. Compliance Roadmap

| Phase | Compliance Target | Actions |
|---|---|---|

| **Phase 3** | HIPAA readiness | BAA with hosting providers (MongoDB Atlas, Railway), formal risk assessment, HIPAA-grade audit logging, data classification labels |

| **U
BAAs

---

## 7. Incident Response Plan (MVP Level)

| Step | Action | Owner | Timeline |

|------

**For MVP**: This is a lightweight, solo-founder plan. For HIPAA phase: formal incident response team, tabletop exercises, and documented procedures required.

---

# File: 06-security-requirements.md

# Phase 6 - Security Requirements

## MediCore HIS - Authentication, Authorization, Encryption & Operational Security

---

## 1. Authentication Requirements

### 1.1 User Authentication

| ID | Requirement | Implementation |

|----|------------|---------------|

| SEC
store

### 1.2 Service-to-Service Authentication

| ID | Requirement | Implementation |

|----|--

### 1.3 Session Management

| ID | Requirement | Implementation |

|----|--

---

## 2. Authorization Matrix

### 2.1 Role-Permission Mapping

| Permission | ORG_ADMIN | FACILITY_ADMIN | DOCTOR | NURSE | RECEPTIONIST | ACCOUNTANT |

|------

**DOCTOR can manage their own schedule only, not other doctors' schedules.*

## 2.2 Authorization Enforcement

```

Requ
```

? PermissionsGuard (check user role has required permission via @Permissions() decorator)

?

**Critical rule**: The `org_id` used in all queries ALWAYS comes from the JWT payload, NEVER from query params or request body. This prevents horizontal privilege escalation.

---

## 3. Encryption Requirements

| ID | Scope | Requirement | Implementation |

|----|--

---

## 4. Input Validation Rules

| Rule | Scope | Implementation |

|------

| Pagination enforced | All list endpoints | Max `limit=100`. Default `limit=20`. Offset-based with total count |

---

## 5. Rate Limiting Strategy

| Endpoint Group | Limit | Window | Response |

|------

Implementation: Redis-backed sliding window counter. Key format: `rate:{endpoint_group}:{identifier}`.

---

## 6. CORS Configuration

```typescript

// Pro

No wildcard origins. No `Access-Control-Allow-Origin: *`. Credentials (cookies) require explicit origin listing.

---

## 7. Security Headers

| Header | Value | Purpose |

|------
```

| `Referrer-Policy` | `strict-origin-when-cross-origin` | Limit referrer leakage |

| `Per

---

# File: 06-threat-model.md

# Phase 6 - Threat Model

## MediCore HIS - STRIDE Analysis per Component

---

### 1. Next.js Frontend

| Threat | Category | Description | Mitigation |

|------

---

### 2. NestJS Core API

| Threat | Category | Description | Mitigation |

|------

| T-API-04 | **Tampering** | Attacker modifies encounter after sign-off | Sign-off sets `status: signed_off` - all subsequent write endpoints reject with 422 for signed-off encounters |

| T-A
CUD
delet

---

## 3. FastAPI AI Sidecar

| Threat | Category | Description | Mitigation |

|------

---

## 4. Database Layer

| Threat | Category | Description | Mitigation |

|------

| T-DB-01 | **Info Disclosure** | Database credentials leaked in source code | All credentials in environment variables. `.env` in `.gitignore`. `env.template` has placeholder values only |

| T-D
enfor

---

## 5. Redis

| Threat | Category | Description | Mitigation |

|------

---

## 6. External Services

| Threat | Category | Description | Mitigation |

|------

---

# File: 07-blueprint.md

# Phase 7 - Execution Blueprint

## MediCore HIS - Agent-Executable Build Steps

> Each step is designed so a fresh developer (or AI agent) can pick it up cold without reading prior steps. Context briefs are self-contained.

---

## Step 1 - Monorepo Scaffolding

**Context:** MediCore is a multi-app monorepo. Three runtime applications (Next.js frontend, NestJS API, FastAPI AI sidecar) share a common TypeScript types package. The monorepo uses pnpm workspaces + Turborepo for build orchestration. Python (FastAPI) lives in the repo but is managed separately with its own `requirements.txt`. Docker Compose provides local development databases.

**Objective:** Create the monorepo skeleton with all configuration files so that `pnpm install && pnpm dev` starts all three apps (even if they only return "Hello World").

**Inputs:** None - greenfield project.

**Outputs:**

```

**Acceptance criteria:**

- `pnp
```

- `packages/shared-types` exports at least one interface importable from both `apps/web` and `apps/api`
- `.en

**Dependencies:** None - first step.

**Estimated time:** 1-2 dev days.

---

## Step 2 - Auth Module

**Context:** MediCore uses JWT-based authentication. NestJS issues access tokens (15-min, in response body) and refresh tokens (7-day, HttpOnly cookie, SHA-256 hashed in PG). The User model lives in PostgreSQL via Prisma. Passwords use bcrypt with cost factor 12. Rate limiting on login: 5 attempts/IP/15 min via Redis.

**Objective:** Implement complete auth flow: register (internal), login, refresh, logout, and `GET /auth/me` with facility roles.

**Inputs:** Prisma schema for User, RefreshToken, UserFacilityRole (from `his-predev/schemas/postgres/schema.prisma`).

**Outputs:**

- `ap
DTOs

**Acceptance criteria:**

- `PO

**Dependencies:** Step 1 (monorepo exists, PG running in Docker).

**Estimated time:** 2-3 dev days.

---

## Step 3 - Tenancy Module

**Context:** MediCore is multi-tenant. Every query is scoped by `org_id` (derived from JWT, NEVER from request body). Organizations have Facilities. The onboarding wizard creates Org + Facility + Admin User atomically in one PG transaction. Prisma middleware auto-injects `org_id` into WHERE clauses. `tenancy.facility.created` event published to Redis.

**Objective:** Implement Organization + Facility CRUD, Prisma tenant middleware, and the atomic onboarding endpoint.

**Inputs:** Step 2 (auth working), Prisma schema for Organization + Facility + Department.

**Outputs:**

- `app

**Acceptance criteria:**

- `PO

**Dependencies:** Step 2 (auth + Prisma setup).

**Estimated time:** 1.5-2 dev days.

---

## Step 4 - Patients Module

**Context:** Patients live in MongoDB (flexible schema for clinical documents). MRN is auto-generated using an atomic PG counter: format `{prefix}-{YYYY}-{counter:05}`. Every patient document carries `org_id` for tenancy scoping. Allergy register is an embedded array with `substance_normalized` field for drug matching. Patient search uses MongoDB text index across `name.display`, `phone`, `mrn`.

**Objective:** Implement patient CRUD: register (with MRN generation), search, get profile, add allergy, update problem list, upload documents to R2.

**Inputs:** Step 3 (tenancy working), Mongoose schema from `his-predev/schemas/mongo/patient.schema.json`.

**Outputs:**

- `app

- Patient search with text index

**Acceptance criteria:**

- `PO

**Dependencies:** Step 3 (tenancy, org_id scoping), Cloudflare R2 configured.

**Estimated time:** 2-3 dev days.

---

## Step 5 - Scheduling Module

**Context:** DoctorSchedule lives in PG (relational - JOINed with Users). Appointments live in MongoDB (document-like - embedded status_history). Available slots are computed by subtracting booked appointments and breaks from the schedule. Double-booking is prevented via Redis optimistic lock (`schedule_lock:{doctorId}:{slot}` with 30s TTL + NX flag). Check-in generates a queue token and updates a Redis Sorted Set. Queue updates stream via SSE.

**Objective:** Implement doctor schedule CRUD, availability computation, appointment booking (with Redis lock), check-in with queue token, SSE queue stream, public queue display endpoint.

**Inputs:** Step 4 (patients exist to book), PG schema for DoctorSchedule, Mongo schema for Appointment.

**Outputs:**

- `app

**Acceptance criteria:**

- `PO

**Dependencies:** Step 4 (patients), Step 2 (auth - DoctorSchedule references User).

**Estimated time:** 2-3 dev days.

---

## Step 6 - Encounters Module (EMR Core)

**Context:** Encounters are the central clinical document in MongoDB. One encounter per appointment (enforced by unique index on `appointment_id`). Vitals are partial-update (autosaved on blur). SOAP notes use debounced autosave (2s client-side). Sign-off validates non-empty Assessment, sets status to `signed_off` (irreversible), and publishes `clinics.encounter.signed_off` event which triggers billing. Prescriptions embed in the encounter document with allergy cross-check. AI fields exist in schema but are populated by the AI module (Step 9).

**Objective:** Implement encounter lifecycle: create, vitals update, SOAP update, diagnosis add (ICD-10), prescription add (with allergy check), sign-off with event publishing.

**Inputs:** Step 5 (appointments exist), Mongoose schema from `his-predev/schemas/mongo/encounter.schema.json`.

**Outputs:**

- `app

**Acceptance criteria:**

- `PO

**Dependencies:** Step 5 (appointments), Step 4 (patients - allergy register).

**Estimated time:** 3-4 dev days.

---

## Step 7 - Billing Module

**Context:** Billing lives in PostgreSQL (financial data needs ACID). Invoices are auto-created when the `clinics.encounter.signed_off` event fires - the billing module subscribes to Redis events. Invoice numbers are auto-generated: `INV-{YYYY}-{counter:05}`. Line items reference the services catalog. Payments support cash (manual recording) and Stripe (payment intents + webhook). Revenue aggregates are computed from PG with GROUP BY facility + date.

**Objective:** Implement invoice auto-creation on sign-off event, invoice CRUD, payment recording (cash + Stripe), Stripe webhook handler, revenue aggregates.

**Inputs:** Step 6 (encounter sign-off events), Prisma schema for Invoice, InvoiceItem, Payment, Service, TaxConfig.

**Outputs:**

- `app

**Acceptance criteria:**

- Sign

**Dependencies:** Step 6 (sign-off events), Step 3 (tenancy - services catalog).

**Estimated time:** 2-3 dev days.

---

## Step 8 - Communication Module

**Context:** MediCore sends SMS via Twilio (appointment confirmation + 24h reminders) and email via SendGrid (invoice PDFs). The communication module subscribes to Redis events and sends messages based on event type. Notification templates live in PG with Jinja2-style variable interpolation. All sent notifications are logged in MongoDB `notification_logs` collection. The 24h reminder uses a Redis delayed job (scheduled at booking time).

**Objective:** Implement SMS sending (Twilio), email sending (SendGrid), notification template system, event-driven message dispatching, 24h reminder scheduling.

**Inputs:** Step 7 (billing events for invoice email), Step 5 (scheduling events for SMS).

**Outputs:**

- `app

**Acceptance criteria:**

- Boo

**Dependencies:** Step 5 (appointment events), Step 7 (invoice events), Twilio + SendGrid accounts.

**Estimated time:** 2 dev days.

---

## Step 9 - AI Service (FastAPI Sidecar)

**Context:** FastAPI is a stateless Python microservice for AI inference. It validates the same JWTs issued by NestJS (shared JWT_SECRET). It never touches databases directly - it calls NestJS `/internal/patients/:id/context` endpoint to get patient data, then constructs prompts using Jinja2 templates and sends them to OpenAI GPT-4o-mini. Rate limiting: 20 requests/user/hour via Redis counter. Response caching: 1 hour via Redis. Three endpoints: `/ai/copilot/summary`, `/ai/copilot/differentials`, `/ai/copilot/investigations`.

**Objective:** Implement the complete AI sidecar with JWT validation, rate limiting, context fetching, prompt engineering, OpenAI integration, response caching.

**Inputs:** Step 6 (encounters + patients exist for context), NestJS internal endpoint.

**Outputs:**

- `app

**Acceptance criteria:**

- `PO

- Request with invalid JWT returns 401

- 21st

**Dependencies:** Step 6 (encounters), Step 4 (patients), OpenAI API key.

**Estimated time:** 2-3 dev days.

---

## Step 10 - Frontend: Auth + Onboarding + Layout

**Context:** Next.js 14 App Router with TypeScript and shadcn/ui component library. Three shell layouts: Admin (org management), Clinical (doctor workflows), Operations (receptionist workflows). Facility switcher in top nav. Pages use React Server Components for data fetching and Client Components for interactivity. Auth state managed via access token in memory + refresh on 401.

**Objective:** Implement login page, onboarding wizard (4 steps), app shell with sidebar navigation, facility switcher, route protection via middleware.

**Inputs:** Step 2 (auth API), Step 3 (onboarding API).

**Outputs:**

- `app

**Acceptance criteria:**

- Log

**Dependencies:** Steps 2-3 (auth + tenancy APIs).

**Estimated time:** 3-4 dev days.

---

## Step 11 - Frontend: Patients + Appointments + Queue

**Context:** The Operations shell contains: patient list with search, new patient form, appointment calendar (FullCalendar component), booking form (modal), queue management with check-in buttons and SSE-powered live updates. The Clinical shell contains: patient list with search, patient profile (demographics, allergies, problem list, encounter history).

**Objective:** Implement all patient-facing and scheduling screens for both receptionist and doctor views.

**Inputs:** Step 4 (patients API), Step 5 (scheduling API), Step 10 (shells).

**Outputs:**

- `app

**Acceptance criteria:**

- Rec

**Dependencies:** Steps 4-5 (APIs), Step 10 (shells).

**Estimated time:** 3-4 dev days.

---

## Step 12 - Frontend: Encounter Editor + AI Copilot

**Context:** The encounter editor is the most complex screen - a 3-column layout: (left) patient summary sidebar with demographics, allergies, problems, recent encounters; (center) encounter form with vitals, SOAP editor, diagnoses, prescriptions; (right) AI copilot sidebar with summary, differential suggestions, investigation suggestions. The center column uses debounced autosave (2s). The AI sidebar loads on encounter open and allows on-demand actions.

**Objective:** Implement the full encounter editor with all sub-forms, autosave, allergy warnings, sign-off flow, and AI copilot integration.

**Inputs:** Step 6 (encounters API), Step 9 (AI API), Step 10 (clinical shell).

**Outputs:**

- `app

**Acceptance criteria:**

- Doc

**Dependencies:** Step 6 (encounters), Step 9 (AI), Step 11 (patient profile).

**Estimated time:** 4-5 dev days.

---

## Step 13 - Frontend: Billing + Analytics + Admin

**Context:** The Operations shell billing pages show invoice lists (filterable by status, date, facility), invoice detail (editable line items for drafts, payment recording), and Stripe payment integration. The Admin shell shows a revenue dashboard (bar chart by facility, line chart trends), team management (user CRUD, role assignment), and audit log viewer. All admin pages are ORG_ADMIN or FACILITY_ADMIN only.

**Objective:** Implement billing screens, payment flows, revenue dashboard, team management, audit log viewer, and organization settings.

**Inputs:** Step 7 (billing API), Step 3 (tenancy API for team management).

**Outputs:**

- `app

- `apps/web/app/(admin)/audit-log/page.tsx` - audit viewer

- `app

**Acceptance criteria:**

- Invo

**Dependencies:** Step 7 (billing), Step 3 (tenancy), Step 10 (shells).

**Estimated time:** 3-4 dev days.

---

# File: 07-blueprint-phase2-3.md

# Phase 2 & 3 - Execution Blueprint

## MediCore HIS - Build Steps 14-35 (Post-MVP)

> Extends `07-blueprint.md` (Steps 1-13). Each step is agent-executable with self-contained context.

---

## Phase 2 Build Steps

### Step 14 - Patient Portal (Auth + Dashboard + Records)

**Context:** The patient portal is a separate Next.js route group `/(portal)` with OTP-based authentication (no passwords). Patients log in with phone number + 6-digit OTP via Twilio. JWT payload includes `role: PATIENT`. Portal shows: dashboard (upcoming appointments, unpaid invoices), encounter history (read-only), prescription downloads. Patient data comes from the existing Patients module - portal endpoints just add a patient-scoped view layer.

**Objective:** Implement OTP auth flow, patient JWT issuance, portal dashboard, medical records viewer, prescription PDF download.

**Outputs:**

- `app

- `apps/web/app/(portal)/` - login, dashboard, records, settings pages

- Red

**Acceptance criteria:**

- Pati

**Estimated time:** 3-4 dev days.

---

### Step 15 - Patient Portal (Appointments + Invoices + Consent)

**Context:** Extends the portal with self-service appointment booking (reuses existing scheduling logic), appointment cancellation, invoice viewing, Stripe payment from portal, and consent management (notification toggles, AI consent, data deletion request).

**Objective:** Complete portal with booking, payments, and privacy controls.

**Outputs:**

- Port

**Acceptance criteria:**

- Pati

**Estimated time:** 3-4 dev days.

---

### Step 16 - Lab Module (Catalog + Orders + Queue)

**Context:** Lab tests are ordered by doctors during encounters, processed by lab techs, and results are authorized by supervising doctors. Lab test catalog lives in PG (name, category, specimen type, normal ranges, price). Lab orders live in MongoDB (flexible result structure). Lab orders auto-create billing charges on authorization.

**Objective:** Implement lab test catalog CRUD, lab order creation from encounters, lab tech queue, specimen tracking, result entry with auto-flagging, authorization workflow.

**Outputs:**

- `app

**Acceptance criteria:**

- Doc

**Estimated time:** 4-5 dev days.

---

### Step 17 - Pharmacy Module

**Context:** Pharmacy has its own drug catalog (PG), inventory with batch/expiry tracking (PG), and dispensing workflow (MongoDB). When a doctor's encounter sign-off fires `pharmacy.rx.received`, the prescription appears in the pharmacist's queue. Pharmacist verifies, checks stock, dispenses, and records. Dispensing triggers billing and stock decrement.

**Objective:** Implement drug catalog, pharmacy inventory, dispensing workflow, stock alerts, purchase orders, supplier management.

**Outputs:**

- `app

**Acceptance criteria:**

- Drug

**Estimated time:** 4-5 dev days.

---

### Step 18 - Telemedicine Module

**Context:** Video consultations use Daily.co API. A video session is created when a telemedicine-type appointment approaches. Patient joins via portal, doctor joins via clinical shell. Doctor can edit encounter during video call (split-screen: video + encounter form). Session end triggers billing for telemedicine service.

**Objective:** Implement Daily.co integration, video room creation, waiting room UI, in-session encounter editing, telemedicine billing.

**Outputs:**

- `app

**Acceptance criteria:**

- Crea

**Estimated time:** 3-4 dev days.

---

### Step 19 - IPD Module (Wards + Beds + Admissions)

**Context:** IPD manages inpatient workflows. Wards and beds live in PG (relational - capacity tracking). Admissions live in MongoDB (document - embedded round notes, medication schedule, bed history). Bed allocation uses the same conflict-prevention pattern as appointment booking (check + lock + write). Daily cron adds bed charges to patient invoice. Discharge generates a summary document and frees the bed.

**Objective:** Implement ward/bed management, patient admission, bed allocation, bed transfers, nursing station dashboard, daily round notes, discharge with summary, bed charge billing.

**Outputs:**

- `app

**Acceptance criteria:**

- Crea

- Transfer patient ? bed history updated ? old bed freed ? new bed occupied

- Nurs

**Estimated time:** 5-6 dev days.

---

### Step 20 - Inventory Module

**Context:** Tracks consumables and equipment (not drugs - those are in Pharmacy). Items have reorder levels. Stock transactions track receive/issue/adjust. Purchase orders have approval workflow. Shares supplier management with Pharmacy.

**Objective:** Implement item catalog, stock transactions, low-stock alerts, purchase orders.

**Outputs:**

- `app

**Acceptance criteria:**

- Item

**Estimated time:** 2-3 dev days.

---

### Step 21 - Radiology Module

**Context:** Similar structure to Lab: orders from encounters, report entry by radiologists, authorization workflow, billing integration. Optional DICOM viewer via Orthanc (iframe or Cornerstone.js).

**Objective:** Implement radiology catalog, order creation, report entry, authorization, optional DICOM viewer.

**Outputs:**

- `app

**Acceptance criteria:**

- Doc

- Radiologist enters report ? authorizes ? billing charge added ? results in encounter

- DIC

**Estimated time:** 3-4 dev days.

---

### Step 22 - Internal Messaging

**Context:** Staff-to-staff messaging within org. Uses MongoDB for messages/conversations and WebSocket for real-time delivery. Conversations can be linked to patient context (so doctors can discuss a case).

**Objective:** Implement conversations, direct messaging, group chats, WebSocket real-time, patient-context linking.

**Outputs:**

- `app

**Acceptance criteria:**

- Sen

**Estimated time:** 3 dev days.

---

### Step 23 - Triage Chatbot + AI Ambient Audio

**Context:** Triage is a public-facing chatbot that collects symptoms, sends them to the AI sidecar for urgency assessment, and optionally books an appointment. Ambient audio captures doctor-patient conversations, sends to Deepgram/Whisper for transcription, then AI structures into SOAP format.

**Objective:** Implement triage chatbot (public, no auth), ambient audio recording + transcription + SOAP auto-population.

**Outputs:**

- `app

**Acceptance criteria:**

- Tria

**Estimated time:** 4-5 dev days.

---

### Step 24 - Platform Features (CMS + i18n + Campaigns + Config + Backup)

**Context:** Bundle of smaller platform features: built-in clinic marketing pages (CMS with section builder), multi-language support (i18n with French/Arabic/English), SMS/email campaigns (bulk outreach), module configuration (toggle modules per org), custom fields, backup/restore admin UI.

**Objective:** Implement all remaining Phase 2 platform features as a single consolidated step.

**Outputs:**

- CMS

**Acceptance criteria:**

- CMS

**Estimated time:** 5-6 dev days.

---

### Step 25 - Helpdesk + Enhanced Analytics + Polish

**Context:** Final Phase 2 step: integrate Tawk.to/Crisp helpdesk widget, build enhanced analytics dashboards (clinical KPIs, demographics, AI usage stats, PDF/CSV export), and polish all Phase 2 screens (loading states, error states, responsive design).

**Objective:** Complete Phase 2 with helpdesk integration, advanced analytics, and UI polish pass.

**Acceptance criteria:**

- Help

**Estimated time:** 3-4 dev days.

---

## Phase 3 Build Steps

### Step 26 - HR Module (Staff + Attendance + Leave)

**Objective:** Staff profiles, attendance tracking (check-in/out), leave management (request ? approve), department-based roster management.

**Esti

### Step 27 - HR Module (Payroll + Reports)

**Objective:** Basic payroll calculation (salary, deductions, bonuses), monthly payslip PDF, payroll reports, staff performance metrics.

**Esti

### Step 28 - Workflow Automation Engine (Simple Rules)

**Objective:** IF/THEN rules engine: define trigger + conditions + actions. Execute on matching events. Execution log with success/fail tracking.

**Esti

### Step 29 - Workflow Automation Engine (Visual Builder)

**Objective:** Drag-and-drop workflow canvas (nodes: trigger ? condition ? action). Template library with pre-built automations. Built with React Flow.

**Esti

### Step 30 - React Native Patient App

**Objective:** Patient mobile app: OTP login, dashboard, records, appointments (book/cancel), invoices (pay), push notifications via Firebase FCM.

**Esti

### Step 31 - React Native Doctor App

**Objective:** Doctor mobile app: login, today's schedule, patient search, simplified encounter editor, AI copilot access, push notifications.

**Esti

### Step 32 - HL7 FHIR API

**Objective:** FHIR R4 resource endpoints for Patient, Encounter, Observation, MedicationRequest, DiagnosticReport, Organization, Practitioner. OAuth2 authentication for external integrations.

**Esti

### Step 33 - AI Advanced (Optimization + Risk Scoring)

**Objective:** Appointment demand forecasting from historical data, population health risk scoring across patient cohort, per-patient longitudinal trend detection (weight, BP, HbA1c).

**Esti

### Step 34 - Ambulance Management

**Objective:** Fleet registry, dispatch linked to patients, ambulance status tracking, ambulance billing charges.

**Esti

### Step 35 - Mobile Polish + App Store Submission

**Objective:** Final polish for both mobile apps, app store screenshots, TestFlight/Google Play internal testing, submission to Apple + Google.

**Esti

---

## Full Timeline Summary

| Phase | Steps | Dev Days | Calendar (15-20h/wk) |

|------

> With a second developer, Phase 2 and 3 could be parallelized significantly (frontend developer + backend developer working simultaneously), reducing total calendar time to ~12 months.

---

# File: 07-definition-of-done.md

# Phase 7 - Definition of Done

## MediCore HIS - What "Done" Means at Every Level

---

## 1. Feature-Level Done

A feature is "done" when ALL of the following are true:

### Code Quality

- [ ] T

### Functionality

- [ ] H

### Testing

- [ ] A

### Security

- [ ] N

### Documentation

- [ ] A

---

## 2. Step-Level Done (Blueprint Step)

A build step is "done" when:

- [ ] All acceptance criteria from the blueprint are met
- [ ] F

---

## 3. Milestone-Level Done

A milestone is "done" when:

### M1 - Foundation Complete

- [ ] S

### M2 - Backend MVP Complete

- [ ] S

### M3 - Full MVP Complete

- [ ] S

### M4 - First Clinic Live

- [ ] P

- [ ] Zero critical bugs reported by clinic staff

- [ ] R

**M5 - First Revenue**

- [ ] S

---

## 4. Project-Level Done (Pre-Dev Package)

The pre-development phase is "done" when:

- [x] Phase 0: Project charter reviewed and signed

- [x] F

---

## 5. Quality Gates (Before Each Milestone)

| Gate | Check | Tool |

|------

| Schema | Database migrations applied cleanly | `npx prisma migrate deploy` |

| Mult

# File: 07-milestones.md

# Phase 7 - Milestones

## MediCore HIS - Build Phases with Deliverables

---

## Milestone Overview

```mermaid

gantt

    section Foundation

Ste

    section Core Backend

Ste

    section Revenue + AI

Ste

    section Frontend

Ste

    section Launch

Int
```

First Clinic Beta :beta, after test, 14d

section Milestones

M1

> **Note**: Timeline assumes ~15-20 hours/week. Backend and frontend steps have limited parallelism (solo developer). Steps 9 and 10 can run in parallel since they're independent.

---

## Detailed Milestones

### M1 - Foundation Complete (Target: Month 1)

| Deliverable | Verification |

|------

**Sign-off**: Oussama verifies all acceptance criteria from Steps 1-3.

---

### M2 - Backend MVP Complete (Target: Month 3)

| Deliverable | Verification |

|------

**Sign-off**: Full end-to-end test: book appointment ? encounter ? AI copilot ? sign-off ? invoice ? payment ? SMS notification.

---

### M3 - Full MVP Complete (Target: Month 5)

| Deliverable | Verification |
|---|---|

**Sign-off**: Oussama uses the system to create a full clinic simulation (10 patients, 10 encounters, AI used, billing recorded).

---

### M4 - First Clinic Live (Target: Month 7)

| Deliverable | Verification |
|---|---|

**Sign-off**: Clinic owner confirms the system is working satisfactorily for daily operations.

---

### M5 - First Revenue (Target: Month 8-9)

| Deliverable | Verification |
|---|---|

**Sign-off**: Bank account shows recurring revenue from MediCore subscriptions.

---

# File: 07-milestones-phase2-3.md

# Phase 2 & 3 - Milestones & Timeline

## MediCore HIS - Full Product Milestones (M5-M10)

> Extends `07-milestones.md` (M1-M4). MVP milestones remain unchanged.

---

## Gantt - Full Product Timeline

```mermaid
gantt

    section Phase 1 (MVP)

Ste

    section Phase 2

Ste

    section Phase 3

Ste
```

section Milestones

M1

---

## M5 - Phase 2 Core Complete (Target: Month 10)

| Deliverable | Verification |

|------

**Sign-off**: Full simulation: OPD visit with lab order ? results ? pharmacy dispensing ? billing ? patient pays via portal.

---

## M6 - Phase 2 Complete (Target: Month 12)

| Deliverable | Verification |

|------

**Sign-off**: Full product demo covering all modules, recorded as video.

---

## M7 - Phase 3 Complete (Target: Month 18)

| Deliverable | Verification |

|------

**Sign-off**: Oussama uses doctor mobile app to complete an encounter with AI copilot, patient pays via mobile app.

---

## Financial Milestones (Updated for Full Product)

```mermaid

graph

style C fill:#4CAF50,color:#fff

sty

---

## Infrastructure Cost Scaling

| Phase | Monthly Cost | Revenue Target | Margin |

|------

---

## Key Scaling Decision Points
```

| Decision Point | When | Options |

|------

---

# File: 07-risk-register.md

# Phase 7 - Risk Register

## MediCore HIS - Risk Identification, Scoring & Mitigation

> Scoring: Probability (1-5) × Impact (1-5) = Risk Score. Scores ?12 = Critical, 8-11 = High, 4-7 = Medium, 1-3 = Low.

---

## Technical Risks

| ID | Risk | Prob | Impact | Score | Mitigation | Owner | Status |

|----|--

## Business Risks

| ID | Risk | Prob | Impact | Score | Mitigation | Owner | Status |
|---|---|---|---|---|---|---|---|

## Regulatory & Legal Risks

| ID | Risk | Prob | Impact | Score | Mitigation | Owner | Status |
|---|---|---|---|---|---|---|---|

---

## Risk Heat Map

```

Impa

**Critical (Score ?12)**: TR-02 (AI accuracy), TR-03 (lost events), BR-01 (sales cycle), BR-02 (burnout)

**Hig

---

## Risk Review Schedule

| Frequency | Action |
|---|---|
```

|----------|--------|

| Eve

---

# File: 08-executive-summary.md

# Phase 8 - Executive Summary

## MediCore HIS - Non-Technical Overview

---

## The Problem

Doctors in private clinics - especially solo practitioners and small polyclinic networks - are drowning in paperwork. For every hour spent with a patient, physicians spend two hours documenting the visit in their records. This "pajama time" (working on notes after the clinic closes) contributes to burnout, reduces the number of patients a doctor can see, and leads to billing errors that leak 5-15% of clinic revenue.

The alternatives available today are either **too expensive** (enterprise systems like Athenahealth at $350-800/month per doctor, requiring 6-12 months to implement) or **too risky** (cheap scripts bought online for $39-199 one-time, but built on outdated technology with no security, no support, and no AI capability).

There is nothing in between: an affordable, modern, intelligent clinic management system.

---

## The Solution

**MediCore** is a cloud-based Hospital Information System that handles the full clinic workflow - from patient registration and appointment scheduling through clinical encounter documentation and billing - with an AI copilot built into the core of the product.

### How It Works (Plain English)

1. **Patient arrives** ? Receptionist registers them in 90 seconds (name, phone, date of birth)

2. **A

5. **AI copilot helps** ? The doctor types shorthand notes; the AI expands them into professional, structured medical documentation (saving 30-45 minutes per day)

6. **E

### What Makes MediCore Different

- **AI-native**: The AI copilot is built into the product from day one - not a bolt-on feature that costs extra

- **Af

---

## The Market

The ambulatory (outpatient) EHR market is worth **$5.7-6.8 billion globally** and growing at 5.4% annually. Clinics are actively moving to cloud systems, with 81% of new EHR deployments being cloud-based.

MediCore initially targets **Morocco and francophone Africa** - a region with thousands of private clinics but almost zero modern, AI-powered cloud options available at an affordable price point.

---

## Timeline & Investment

| Phase | What Gets Built | When |

|------

**Total investment to first customer**: ~$272 in infrastructure costs + ~500 development hours.

**Break-even**: 3 solo clinic subscriptions ($39/mo × 3 = $117/mo, covering all hosting costs).

---

## Revenue Potential

| Timeframe | Target | Projected ARR |

|------

| Year 1 | 5-10 clinics | $6,000-8,000 |

| Yea

---

## Next Step

This document is part of a comprehensive pre-development package that includes market research, technical architecture, database design, API specifications, security analysis, and a detailed build plan. The system is designed and ready to build.

**The next action is: start Step 1 - scaffold the monorepo and begin coding.**

---

# File: 08-project-proposal.md

# Phase 8 - Project Proposal

## MediCore HIS - Full Project Proposal

---

## 1. Executive Summary

MediCore is a multi-tenant SaaS Hospital Information System designed for solo practitioners and polyclinic networks. It combines clinical operations management (patients, scheduling, encounters, billing) with an AI copilot that transforms shorthand clinical notes into structured medical documentation - reducing physician documentation time by 30-45 minutes per day.

The product targets an underserved segment: affordable ambulatory clinics in Morocco, MENA, and francophone Africa that need enterprise-grade functionality at a fraction of enterprise pricing.

---

## 2. Scope of Work

### 2.1 What Will Be Delivered (MVP)

| Module | Capabilities |

|------

| **Patient Management** | Registration with auto-MRN, search (name/phone/MRN), allergy register, problem list, document uploads |

| **S
mana

### 2.2 What Is Explicitly Excluded (Phase 2+)

| Excluded Feature | Phase |

|------

---

## 3. Technical Architecture

### Stack

| Layer | Technology |

|------

| Document DB | MongoDB 7.0 (patients, encounters, appointments) |

| Cac

### Key Architectural Decisions

- Eve

Full C4 diagrams, data flow diagrams, ERD, and ADRs are available in the `pre-dev/` documentation package.

---

## 4. Timeline & Milestones

| Milestone | Target | Deliverables |

|------

**Developer capacity**: 15-20 hours/week (solo founder, part-time alongside studies).

---

## 5. Team & Roles

| Role | Person | Responsibility |

|------

---

## 6. Investment Breakdown

### Development Phase (Months 1-8)

| Item | Cost |
|---|---|

### Operational Phase (Month 8+)

| Item | Monthly Cost |
|---|---|

### Revenue Targets

| Period | MRR Target |
|---|---|

---

## 7. Pricing Model

| Tier | Price | Target |
|---|---|---|

---

## 8. Risk Summary

| Risk Category | Top Risk | Mitigation |
|---|---|---|

|--------------|----------|------------|

| **Te

Full risk register with probability/impact scoring and mitigations is in `pre-dev/07-risk-register.md`.

---

## 9. Success Criteria

The project is considered successful if, by Month 12:

1. ? Working MVP used by at least 1 clinic daily

2. ? A

---

## 10. What Happens Next

1. **Immediate**: Begin Step 1 - monorepo scaffolding

2. **V

---

# File: 08-sign-off-checklist.md

# Phase 8 - Sign-Off Checklist

## MediCore HIS - Pre-Development Review & Approval

---

## Client Sign-Off

**Project:** MediCore HIS - Multi-Tenant SaaS Hospital Information System

**Pre

---

I confirm I have reviewed and approved the following deliverables:

**Phase 0 - Project Charter**

- [ ] P

**Phase 1 - Discovery & Research**

- [ ] M

**Phase 2 - Product Requirements**

- [ ] P

**Phase 3 - System Architecture**

- [ ] T

**Phase 4 - UX & Information Architecture**

- [ ] S

- [ ] 25 screens is the right scope for MVP _____

**Phase 5 - Data & API Design**

- [ ] E

**Phase 6 - Security & Compliance**

- [ ] S

**Phase 7 - Execution Blueprint**

- [ ] 1

**Phase 8 - Client Package**

- [ ] E

---

## Locked Decisions Summary

| Decision | Choice | Reference |

|------

| Multi-tenancy model | Shared instance, org_id scoping | ADR-005 |

---

## Signatures

**Product Owner / Founder:**

Name: Oussama Zaki

Title:

---

**Project Lead / Technical Architect:**

Name: Oussama Zaki

Title:

---

> **Note**: For a bootstrapped solo project, both signatures are the same person. When investors, co-builders, or contractors join, this sign-off becomes a formal agreement between parties. The pre-development package above this checklist is the definitive scope reference.

---

# File: 08-statement-of-work.md

# Phase 8 - Statement of Work

## MediCore HIS - Scope, Deliverables & Terms

---

## 1. Parties

| Role | Entity | Representative |

|------

> This SOW documents the founder's commitment to himself and future stakeholders. It serves as a binding scope reference for any co-builders, contractors, or investors who join later.

---

## 2. Services to Be Delivered

### 2.1 Deliverable 1 - Pre-Development Package

| Item | Status |
|---|---|

### 2.2 Deliverable 2 - MVP Software

| Module | Scope | Acceptance Criteria |
|---|---|---|

### 2.3 Deliverable 3 - Production Deployment

| Item | Criteria |
|---|---|

| PostgreSQL on Railway | Daily backups, connection pooling |

| Mon

---

## 3. Acceptance Criteria (System-Level)

The complete system is accepted when:

1. A new clinic can onboard via the wizard in <10 minutes, unassisted

2. A r

---

## 4. Change Request Process

| Situation | Process |

|------

---

## 5. Payment Milestones

> Not applicable for bootstrapped solo project. This section is included for future reference if external funding or contractors are involved.

| Milestone | Payment Trigger | Amount |

|------

---

## 6. Intellectual Property

| Asset | Ownership |

|------

**Data processing relationship**: EPIOSO Studio (MediCore) is the **data processor**. Each clinic organization is the **data controller**. A Data Processing Agreement (DPA) will be provided to each clinic upon subscription.

---

## 7. Confidentiality

- All patient data is confidential and subject to GDPR-style data protection

- Sou

---

## 8. Termination

| Scenario | Outcome |

|------

---

# File: ADR-001-nestjs-over-fastapi.md

# ADR-001 - NestJS over FastAPI for Main Backend

**Status**: Accepted

**Dat

---

## Context

The original blueprint used FastAPI (Python) for all backend logic. After reflection on the full product scope, the stack was reconsidered.

## Decision

Use NestJS (TypeScript) as the primary backend for all HIS business logic. FastAPI is retained only as a dedicated AI microservice.

## Rationale

| Factor | FastAPI (original) | NestJS (chosen) |

|---|---

The strongest argument: TypeScript across Next.js + NestJS means shared DTOs, shared validation schemas (Zod), and one `packages/shared-types` package that both apps import. When a field changes in an API response, the frontend catches the type error immediately.

## Consequences

- FastAPI is now an AI-only sidecar service. It has no database access.

- Nes

---

# File: ADR-002-hybrid-database.md

## ADR-002 - Hybrid Database: PostgreSQL + MongoDB + Redis

**Status**: Accepted

**Dat

### Decision

Use P
data.

### Rationale

See `

### Risk

Adde
Mong

---

## ADR-003 - FastAPI as AI Sidecar (Not Integrated into NestJS)

**Status**: Accepted

**Dat

### Decision

FastA
conne

### Rationale

Pytho
Type
non-r

The sidecar pattern keeps the boundary clean: NestJS owns data, FastAPI owns inference.

### Communication pattern

```

### Security

FastA

The `/internal/*` NestJS routes (used only by FastAPI) require an additional `X-Internal-Service-Key` header that's only known to FastAPI. These routes are never exposed publicly.

---

## File: ADR-004-005-events-and-tenancy.md

## ADR-004: Redis Pub/Sub over Dedicated Message Queue

**Status:** Accepted

**Dat

### Context

Medi
transp

### Decision

Use R

### Rationale

| Fact
```

| Scale needed | 50 users | 50 users | 500+ users | 10,000+ users |

At MVP scale (50 concurrent users, ~100 events/hour), Redis pub/sub is sufficient. The risk of lost events is low and the consequence is a missed invoice draft - catchable by a daily reconciliation cron job.

### Alternatives Considered

- **Bu
reliab

### Consequences

- **Ea

---

## ADR-005: SaaS Multi-Tenant over Single-Tenant Instances

**Status:** Accepted

**Dat

### Context

Medi
`org_

### Decision

SaaS
`facili

### Rationale

| Fact

| Data isolation | Application-level (Prisma middleware + Mongoose guards) | Infrastructure-level (separate DBs) |

| Com

For a solo builder, managing separate infrastructure per customer is operationally impossible. Multi-tenancy trades compliance simplicity for operational feasibility.

### Isolation Enforcement

1. *
`01-a

### Consequences

- **Ea

---

## File: PROGRESS.md

## Pre-Dev Progress

**Project:** MediCore HIS

**Sta

### Phase 1 (MVP) - Original 8 Phases

| Phase | Status | Files |

|------

| 5 - Data & API | ? Complete | `05-ERD.md`, `05-database-schema.md`, `05-data-flow.md`, `05-api-contracts/openapi.yaml` |

| 6
`06-p

## Phase 2/3 Expansion (Full Product Roadmap)

| Document | Status | File |

|------

## Design System

| Document | Status | File |

|------

## Pages Bible (Full Page Audit)

| Document | Status | File |

|------

| Part 5: IPD, HR, Portal, Settings, Relations | ? Complete | `04-pages-bible-part5.md` |

## Summary Statistics

| Metric | Phase 1 | Phase 2/3 | Total |
|---|---|---|---|

## Locked Decisions

- AI C

## Next Action

Start

---